



华北电力大学

毕业设计（论文）

论文题目： 基于机器学习的无人机博弈对抗研究

院 系： 控计学院

专 业： 计算机科学与技术

班 级： 计算2202

姓 名： 蒋伦吉

学 号： 120221080514

指导教师： 刘春阳

2026年5月

摘 要

强化学习是机器学习及人工智能领域比较重要的研究方向与工程方法，用于解决序列决策问题，近年来特别是在游戏，机器人路径规划，策略博弈等场景中被广泛使用。机器博弈研究多个智能体在环境中的对抗与合作行为，通过交互学习最优策略，在实时策略博弈、机器人集群协作等复杂决策场景中具有重要的研究价值与应用前景。

本文基于 Godot 游戏引擎开发构建了 2D 俯视角多智能体混战竞技场博弈环境《Tiny Swords》，设计并实现了包含自身状态、对手信息、游戏内元素感知与 LiDAR 射线检测障碍物的 174 维分段式观测空间，并为每个智能体单独设计了游戏事件的奖励函数，制定了博弈规则。进一步地为了解决奖励稀疏的问题，采用了小事件稠密引导奖励和基于势能的奖励塑形方法，提高了策略收敛速度。在此基础上采用独立近端策略优化算法，先在单智能体环境下通过课程学习让智能体从易到难地学习期望行为，同时进行了必要的消融实验，对比与讨论了在该特定环境下势函数形式、障碍物处理策略与网络架构对智能体基础行为的影响。

此后将训练好的智能体放在多智能体环境中学习博弈与对抗，针对多智能体训练中的非平稳性与违反单智能体强化学习算法假设问题，参考了在对称博弈中常用的联盟训练方法和虚拟自博弈方法，针对本文的特定非对称对抗环境，简化为了对手池训练方法和虚拟博弈方法，同时考虑了在多智能体协作型任务中表现优良的独立近端策略优化算法，将这三种方法进行对比测试。

关键词：强化学习；独立近端策略优化；博弈对抗；奖励塑形；循环神经网络

ABSTRACT

With the continuous success of reinforcement learning (RL) in complex decision-making domains, multi-agent reinforcement learning (MARL) has emerged as a critical frontier for studying intelligent behavior in interactive environments. A central challenge in MARL is the non-stationarity introduced by concurrently learning agents, which violates the assumptions of single-agent RL algorithms and degrades training stability. While the centralized-training-decentralized-execution (CTDE) paradigm has proven effective in cooperative settings, its reliance on global information sharing is often infeasible in adversarial scenarios.

This thesis constructs a 2D top-down four-player battle-royale arena using the Godot game engine, featuring differentiated agent roles (warrior, assassin, knight, ranger), multi-source rewards (ball collection, combat, obstacles), and a curriculum learning pipeline with progressively increasing complexity. Building on this environment, a two-phase training framework is proposed: single-agent curriculum-based PPO training with systematic ablation studies, followed by multi-agent adversarial training combining Independent PPO (IPPO) with a Prioritized Fictitious Self-Play (PFSP) opponent pool. The ablation studies cover observation design (valid masks), reward shaping (potential-based functions for ball guidance), obstacle avoidance strategies, and network architecture comparisons (MLP vs. Segmented MLP vs. GRU-MLP), establishing an optimal configuration for the subsequent multi-agent phase.

This work provides a reproducible experimental platform and a systematic training methodology for studying emergent adversarial dynamics in multi-agent game environments.

KEY WORDS: Multi-Agent Reinforcement Learning; Independent PPO; Game-Theoretic Adversarial; Reward Shaping; Recurrent Neural Network

目 录

目 录

1 绪论	1
1.1 研究背景与意义	1
1.2 国内外研究现状	2
1.2.1 强化学习	2
1.2.2 博弈环境与训练平台	3
1.2.3 对抗训练与自博弈	4
1.2.4 奖励工程	5
1.3 论文研究内容	5
1.4 论文组织结构	6
2 相关理论与技术基础	6
2.1 强化学习	6
2.1.1 马尔可夫决策过程	7
2.1.2 部分可观测马尔可夫决策过程	8
2.1.3 近端策略优化算法	8
2.2 多智能体强化学习	9
2.3 奖励塑形理论	10
2.4 Godot 引擎与 RL 集成框架	10
3 博弈环境设计与实现	10
3.1 游戏总体设计	10
3.2 观测空间设计	11
3.3 奖励函数设计	13
3.4 课程学习环境	14
4 单智能体基础训练与对比实验	15
4.1 训练框架	15
4.2 观测空间消融：有效掩码	16
4.3 奖励球塑形方案对比	16
4.4 障碍物避障策略	17
4.5 网络架构对比	21
5 多智能体博弈方法探索	22
5.1 对手池初始化	24
5.2 待评估智能体的训练	25
5.3 实验结果与讨论	25
6 总结与展望	26
6.1 工作总结	26
6.2 未来展望	27
结论	27
A 附录A 实验补充数据	32
致谢	35

第1章 绪论

1.1 研究背景与意义

强化学习（Reinforcement Learning, RL）是机器学习的重要分支，通过试错与环境交互学习最优策略，已在复杂决策问题中展现出巨大潜力——从 AlphaGo^[1] 在围棋中超越人类顶尖棋手，到 AlphaStar^[2] 在星际争霸II中达到宗师级水平，再到 OpenAI Five 在 Dota 2 中击败世界冠军队伍^[3]。然而如何成功地应用强化学习是一系列工程性问题：如何从零设计并实现一个博弈环境，并提供恰当的强化学习算法接口，如何将游戏状态编码为适合神经网络输入的观测向量，如何设计奖励函数以引导智能体逐步习得避障和战斗等基础技能，如何在部分可观测条件下选取合适的网络结构以平衡时序记忆与训练效率。本文针对这些问题给出了一套实践经验。

AlphaGo等里程碑不仅展示了 RL 的技术高度，更揭示了一个深层规律：AI 的重大突破往往发生在博弈对抗场景中。博弈对抗能成为 AI 研究的试金石，在于它天然要求智能体具备适应性决策能力。在博弈场景中，智能体不仅需要利用环境资源、遵循游戏规则，更需要感知对手策略并作出动态调整——“最优”策略并非静态存在，而是相对于对手策略的函数。从早期的 Deep Blue^[4] 到近年来的 AlphaStar，AI 博弈能力每一次提升的背后，都是对对手建模、策略适应与对抗进化等核心问题的深入回应。与完全信息的棋盘博弈不同，实时策略博弈具有状态空间庞大、信息不完全、决策异步等特点，更贴近真实世界的决策场景。

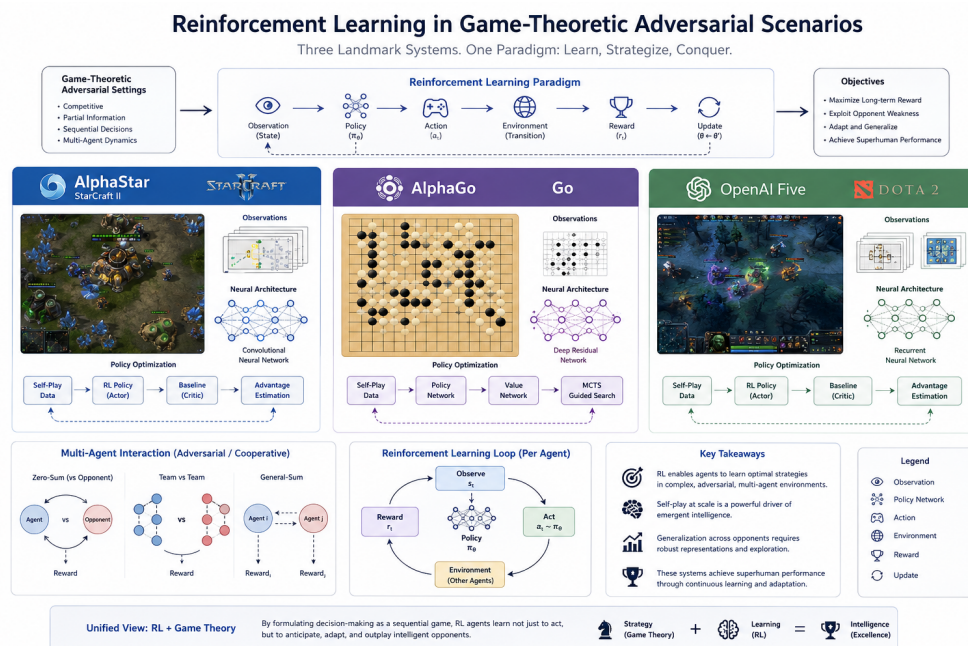


图 1-1: 强化学习博弈应用场景示意

良好的博弈能力体现在智能体面对各种各样的对手时都有不错的表现，为了体现对手的丰富性，本课题将强化学习智能体作为对手，在多智能体博弈中探索智能体如何能够应对纷繁复杂，行为各不一致的对手。当多个自学习智能体共处同一博弈环境时，问题从单智能体优化升级为博弈动力学问题。多智能体强化学习（Multi-Agent RL, MARL）面临的核心挑战是非平稳性（non-stationarity）：每个智能体的策略更新都会改变其他智能体面对的环境转移概率，使单智能体 RL 中的 Markov 决策过程（MDP）假设不再成立^[5]。多智能体强化学习的训练范式按其信息利用方式可大致划分为三类：集中式训练集中式执行（CTCE）、集中式训练分布式执行（CTDE）和分布式训练分布式执行（DTDE）。

CTCE 范式假设训练和执行阶段均存在一个中心控制器，能够获取所有智能体的全局信息并输出联合动作。这一假设在多数实际场景中不成立，更常见的做法是采用 CTDE 范式：训练时利用全局信息学习 Critic 或价值函数，执行时每个智能体仅依赖局部观测做决策。代表性方法包括 QMIX^[6] 和 MAPPO^[7]。这些方法通过集中式 Critic 缓解多智能体环境的非平稳性问题，在协作型任务上有很好的表现。

然而，CTDE 范式引入集中式 Critic 的前提是训练阶段允许全局信息共享，本文是完全对抗的混战环境，基于智能体之间无法共享任何信息的假设，将保持执行去中心化与对手信息隔离，选择 DTDE 作为训练方式。DTDE 的一种典型实现是独立 PPO（Independent PPO, IPPO）：每个智能体独立运行 PPO 算法，将其他智能体视为环境的一部分，不引入任何集中式组件。de Witt 等人^[8]的研究表明，在 SMAC 协作任务上，IPPO 能够在 SMAC 协作任务中超越 QMIX 方法。其成功很大程度上源于 PPO 算法所固有的策略更新约束——通过裁剪机制限制新旧策略之间的差异，使得训练过程本身较为平稳，在一定程度上克服了多智能体环境非平稳性对单智能体算法的不利影响。

本文首先通过实验对比，给出了一套适用于所构建博弈环境的强化学习训练方案，并将其运用于单智能体训练中；在智能体习得基础技能之后，本文进一步将其置于多智能体对抗环境中，以 IPPO 独立训练为基线，实现一种简化的 PFSP 对手池训练方案，对比两种模式下的策略表现，作为对抗型任务中独立训练与虚拟博弈差异的初步探索。

1.2 国内外研究现状

1.2.1 强化学习

强化学习算法主要可分为两类，即基于价值的方法和基于策略的方法。基于价值的方法通过计算状态-动作对的价值函数 Q ，在每个状态下 Q 值最大的动作就是智能体应该采取的动作，目前主流的基于价值的算法思想是 Q 学习（ Q -Learning）^[9]，其求解过程本质上是在直接求解贝尔曼最优方程；基于策略的方法直接求解最优策略函数 $\pi(a|s)$ ，通过构造合适的优化目标，并采用梯度下降（上升）法进行优化；因此，强化学习算法有两条路线，一是求解最优价值间接求解最优策略，二是直接求解最优策略，而目前最主流的强化学习算法思想是将两者结合的演员-评论家模型（Actor-Critic），在算法中会同时求解价值和策

略，用估计的价值指导策略的更新。

强化学习算法通常与深度学习结合使用，这归功于神经网络极其优秀的拟合性能^[10-11]，无论面临如何复杂的环境，神经网络都能够拟合动作值（Q-value）函数，利用神经网络拟合Q值的DQN算法训练出的智能体性能在多款雅达利游戏中可以超越人类^[12]，而在演员-评论家模型中，策略函数和优势函数也都可以用神经网络方便地拟合，这使得深度强化学习成为强化学习中的主要范式。

近端策略优化算法（Proximal Policy Optimization, PPO）^[13]是当前应用最广泛的强化学习算法之一。PPO使用广义优势估计（Generalized Advantage Estimation, GAE）^[14]配合重要性采样有效利用样本来计算策略梯度，同时裁剪新旧策略之间的重要性比例约束策略更新幅度，在保持训练稳定性的同时兼顾样本效率，且对超参数相对鲁棒，使其成为复杂任务中的首选算法。近年来，PPO及其改进版本在游戏AI、路径规划、自动驾驶和机器人控制等领域取得了广泛成功^[15-17]，展现出从虚拟环境策略到现实世界应用的可迁移潜力。

同时，有诸多训练方法用于提高智能体训练效率，比如分层强化学习^[18]，课程学习^[19]，策略迁移^[20]等。其中分层强化学习将一个复杂目标分解为多个阶段性简单子目标，每个子目标都用MDP进行建模，并分配给不同的智能体进行分层训练和执行，每个智能体专注于自身目标，相比于一个智能体解决整个任务，训练难度能显著降低。课程学习指在训练过程中，通过逐渐增加任务难度，使智能体不断适应更复杂的环境。通常课程的设计采用逆向生成，即从目标的最终状态开始，逐步分解出和上一个状态相近但是又在远离最终状态并靠近初始状态的状态，智能体将从最终状态逆向地学习在每个状态的策略。策略迁移则通过将已学策略迁移到新任务中，改进从头训练慢，样本获取率和利用率低等问题。

1.2.2 博弈环境与训练平台

强化学习的研究进展与实验平台的发展密不可分。在单智能体领域，Arcade Learning Environment（ALE）^[21]和OpenAI Gym^[22]提供了标准化的基准测试接口，推动了DQN、PPO等算法的快速发展。在多智能体领域，SMAC^[23]和MPE（Multi-Agent Particle Environment）^[24]被广泛采用，前者基于星际争霸II引擎提供协作战斗场景，后者以简化的二维粒子环境支持协作与竞争任务研究。

上述标准化基准环境在降低入手门槛的同时也限制了研究的多样性——其观测空间与奖励函数通常由环境提供者预先定义，研究者在环境设计层面的探索空间极为有限。这种限制在博弈对抗研究场景中尤为突出：博弈策略的丰富性依赖于精心设计的对抗规则、差异化角色定位和各类环境交互元素的综合配置，仅依赖现有基准环境难以满足定制化需求。

近年来，游戏引擎与RL的集成框架为定制环境的快速开发开辟了新路径。Unity ML-Agents^[25]使开发者能够将Unity引擎中的游戏环境包装为标准的Gym训练环境，

并通过 gRPC 通信机制与 Python 端进行高效对接。Beeching 等人开发的 Godot RL Agents^[26] 同样提供了标准化的 Gym 环境接口和多种环境包装器，内置基于 TCP 的通信机制与 Python 训练端建立连接，在开源游戏引擎 Godot 上实现了训练智能体的便捷方案。与 Unity ML-Agents 相比，Godot RL Agents 依托 Godot 引擎的 MIT 开源协议，完全免费且无授权限制；其轻量化的架构设计和原生 GDScript 支持为学术研究中快速迭代环境原型提供了更具成本优势的选择。

本文选择 Godot RL Agents 作为底层集成框架，将工程重心置于博弈规则设计、观测工程、基于 PBRs 的多层奖励系统与课程学习流程等更高层次的强化学习工程问题上，以此构建面向多智能体博弈对抗的定制化博弈系统。

1.2.3 对抗训练与自博弈

如果直接让两个智能体进行对抗并同时更新策略，那么对于其中任何一个智能体，由于另一个智能体的策略在变化，因此对其来说环境就在变化，即环境是不平稳的，单智能体 RL 的 Markov 决策过程假设不再成立。为了让智能体通过与对手对抗来提升技能，最直接的方式是自博弈（Self-Play），比如 AlphaGo^[1] 通过不断与自身的历史版本对弈生成训练数据，结合蒙特卡洛树搜索（MCTS），在围棋这一两人零和完全信息博弈中达到了超人类水平，展示了自博弈在驱动策略进化方面的巨大潜力。

然而，自博弈会出现策略循环和过拟合问题，智能体尽管在不断地战胜上一个状态的自己，但有可能是在石头剪刀布的圈子里不断循环。于是通常会使用虚拟自我博弈（Fictitious Self-Play, FSP）来进行训练，这一方法基于经典博弈论中的虚拟博弈^[27]，FSP 的对手不是自己的副本或最新的策略，而是自己历史策略的平均值，从而学到一个不会轻易被对手利用，更加鲁棒的策略，通常平均策略是用神经网络表达，即神经虚拟自我博弈（Neural Fictitious Self-Play, NFSP）^[28]。

以上方法都是采用自博弈绕过了多智能体博弈时环境不平稳问题，但是虚拟博弈技巧也不仅仅局限于自我博弈，在星际争霸 II 中能力达到宗师级别的强化学习智能体 AlphaStar^[2] 训练过程中，DeepMind 团队采用了名为联盟训练的技术，一个联盟由 3 类智能体组成：Main Agents, Main Exploiter 和 League Exploiter，每类智能体训练时，其他智能体的策略冻结。其中 Main Agents 的目标是不断训练成为联盟中的最强策略，能够战胜联盟内的所有对手的策略；Main Exploiter 每次从初始状态出发与 Main Agents 进行博弈，专门负责寻找 Main Agents 的策略漏洞，每次训练完成后 Main Exploiter 的策略会重置，每次从新的角度寻找 Main Agents 的策略漏洞；League Exploiter 直接和联盟内所有对手博弈，每次训练后有一定几率重置。

进一步地，PSRO（Policy Space Response Oracles）框架^[29] 将上述思路推广为统一的理论框架：每个智能体维护一个策略池，每轮迭代对池中策略的元策略混合求近似最优响应，将新策略加入池中后继续下一轮。PSRO 在有限博弈中具有收敛到纳什均衡的理论保证，为理解不同自博弈变种的非平稳性处理能力提供了统一视角。

1.2.4 奖励工程

奖励工程（Reward Engineering）是强化学习中决定智能体行为质量的核心环节，直接影响了学习效率和最终性能^[30]。目前主要包含以下核心方法：

势能奖励塑形（PBRs）。奖励塑形通过在环境奖励基础上引入额外的塑形奖励 $F(s, a, s')$ 来加速学习，但任意添加塑形奖励可能改变最优策略。Ng 等人^[31]提出的 PBRs 理论证明，当塑形奖励具有 $F(s, a, s') = \gamma\Phi(s') - \Phi(s)$ 的形式时，不改变 MDP 的最优策略。这为游戏 AI 中的引导奖励设计提供了严格的理论基础——设计者可以通过构造势函数 $\Phi(s)$ 将领域知识注入学习过程，以稠密的引导信号替代稀疏的终局奖励，在保持最优解不变的前提下显著加速收敛。

在此基础上，研究者发展出多种 PBRs 变体以适应更复杂场景。动态势能奖励塑形（DPBRs）^[32]将时间维度引入势函数 $\Phi(s, t)$ ，使塑形信号随训练进度自适应调整，在多智能体系统中能够促进协作与协调。针对多智能体场景，Devlin 等人^[33]进一步证明 PBRs 在随机博弈中不改变纳什均衡，但会通过影响探索过程导致收敛到不同的联合策略。

逆强化学习（Inverse Reinforcement Learning, IRL）。与手动设计奖励函数不同，逆强化学习从专家示范中推断隐含的奖励函数。Ng 和 Russell^[34]最早提出 IRL 框架，后续发展了最大熵 IRL^[35]、最大熵深度 IRL^[36]以及基于模型的 IRL^[37]等方法。IRL 的核心优势在于避免了人工设计奖励函数的困难，但需要高质量的专家示范数据，且推断出的奖励函数可能存在歧义——多个不同奖励函数都能解释同一组示范行为。Hadfield-Menell 等人^[38]进一步提出逆奖励设计（Inverse Reward Design）框架，使智能体能够在奖励函数存在不确定性的情况下进行鲁棒的策略学习。

内在奖励与好奇心驱动。为缓解稀疏奖励环境中的探索困难，研究者提出多种不依赖环境奖励的内在动机机制。Pathak 等人^[39]的 ICM 模块通过预测动作后果的状态变化来生成探索奖励，鼓励智能体访问预测误差大的陌生状态；Burda 等人^[40]的 RND 使用随机网络蒸馏的预测误差作为探索信号，在蒙特祖玛的复仇等极度稀疏奖励的雅达利游戏中取得突破。Devidze 等人^[41]进一步将探索与奖励塑形统一，利用探索过程的经验指导 PBRs 势函数的自动构造，为两类方法的融合提供了新思路。

基于大语言模型的奖励生成。最新的研究方向利用大语言模型（LLM）的常识与推理能力自动生成奖励函数。Text2Reward^[42]通过 LLM 理解自然语言任务描述，生成稠密的、可执行的 Python 奖励代码，在机器人操作等任务中验证了自动奖励设计的可行性。这一方向为降低奖励工程中的人工设计成本、提升跨任务泛化能力提供了全新范式。

1.3 论文研究内容

本文的主要研究内容包括：

（1）构建了基于 Godot 引擎的 2D 俯视角多智能体竞技场博弈环境《Tiny Swords》，设计了 174 维分段式观测空间与 LiDAR 射线检测机制，实现了 4 人混战游戏模式，支持课

程学习与 per-player 差异化奖励配置。

(2) 设计了包含奖励球势能引导、战斗奖励塑形与事件稀疏奖励的多层奖励系统，通过系统的消融实验确定了最优的势函数形式（线性函数、最近球模式）与障碍物处理策略（稀疏惩罚）。

(3) 探讨了针对该特定环境的时序网络使用方案，将门控循环单元GRU引入了强化学习算法，对比了在构建的该特定环境下 MLP、Segmented MLP 与 GRU-MLP 三种网络架构在拾取奖励球任务与战斗任务中的性能差异，为后续更高效的训练提供参考。

(4) 对比了独立近端策略优化与对手池博弈训练框架的结果。先通过冻结对手策略、轮换训练和基于历史对战表现的优先采样等方式驱动智能体在竞争中持续进化，初始化对手池，然后再进行比较评估。

1.4 论文组织结构

本文共分为六章，各章内容安排如下：

第一章介绍研究背景、国内外研究现状、研究内容与论文组织结构。

第二章阐述相关理论与技术基础，包括强化学习（MDP、Bellman 方程、PPO）、多智能体强化学习（IPPO、非平稳性）、奖励塑形理论（PBRs）与 Godot-RL 集成框架。

第三章详细描述博弈环境的设计与实现，包括游戏规则、观测空间、奖励函数与课程学习环境的完整设计。

第四章介绍单智能体基础训练与消融实验，涵盖奖励球势函数对比、障碍物避障策略、网络架构对比与观测空间消融等实验。

第五章对比IPPO和对手池博弈的训练结果。

第六章总结全文工作，展望未来研究方向。

第2章 相关理论与技术基础

2.1 强化学习

强化学习是一门关于个体进化的科学，能够用于解决序列决策问题，智能体与环境不断交互试错并改进自身策略，逐步获取更高的奖励。强化学习过程由两部分组成：智能体和环境。智能体在环境中进行观测，并根据观测结果选择一个动作执行，环境接受智能体的动作并做出响应，包括环境的状态变化和给予智能体的奖励信号，智能体根据新的观测和上一步动作的奖励改进自身行为策略，以期在长期内获得最大的累积奖励^[43]。图 2-1描述了强化学习的基本框架。

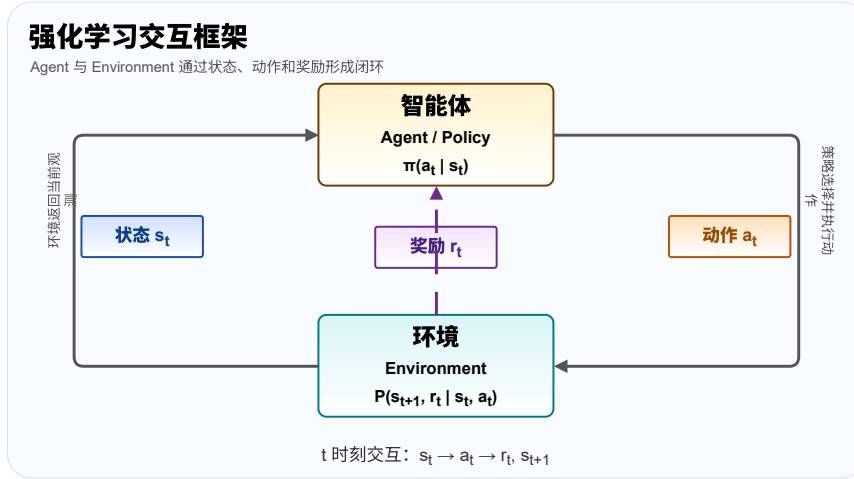


图 2-1: 强化学习基本框架

2.1.1 马尔可夫决策过程

马尔可夫决策过程（Markov Decision Process, MDP）相比于马尔可夫过程增加了动作因素，用于在系统状态空间具有马尔可夫性质的环境中描述智能体的状态，动作和奖励的关系，用五元组 $\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma \rangle$ 表示，其中 $\mathcal{S}$ 为状态空间， $\mathcal{A}$ 为动作空间， $P(s'|s, a) : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \rightarrow [0, 1]$ 为状态转移概率， $R(s, a) : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ 为奖励函数， $\gamma \in [0, 1]$ 为折扣因子。MDP 依赖于马尔可夫性质，即状态转移概率只取决于当前状态和执行的动作，与过去的状态无关，即 $P(s_{i+1}|s_i, a_i) = P(s_{i+1}|s_i, a_i, s_{i-1}, a_{i-1} \cdots s_0, a_0)$ 。轨迹是指 MDP 上的样本轨道，可定义为：

$$\tau = (s_0, a_0, r_0, s_1, a_1, r_1, \cdots, s_T, a_T, r_T) \quad (2-1)$$

求解 MDP 模型是要找到一条轨迹能够最大化累计折扣回报：

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \quad (2-2)$$

其中 γ 代表了智能体对于未来奖励的重视程度， γ 越接近 0，表示智能体越看重眼前的奖励； R_{t+k+1} 为 t 时刻到 $t+k+1$ 时刻的奖励。

值函数 $V^\pi(s)$ 表示从状态 s 出发，遵循策略 π 的期望回报；动作值函数 $Q^\pi(s, a)$ 则表示在状态 s 采取动作 a 后的期望回报。设策略函数为 $\pi(a|s)$ ，动作值函数和值函数分别定义为：

$$Q^\pi(s, a) = R(s, a) + \gamma \sum_{s'} P(s'|s, a) V^\pi(s') \quad (2-3)$$

$$V^\pi(s) = \sum_a \pi(a|s) \left[R(s, a) + \gamma \sum_{s'} P(s'|s, a) V^\pi(s') \right] = \sum_a \pi(a|s) Q^\pi(s, a) \quad (2-4)$$

式 2-4 中的 $V^\pi(s')$ 可以根据轨迹继续递归求解,从而可以得知 MDP 的求解就是要找到一个策略 π^* , 使得对于每一个状态 s , $V^\pi(s)$ 达到最大值 $V^{\pi^*}(s)$, 基于价值的强化学习算法就是基于这种思路, 将最优策略的求解转化为准确地计算 Q 值, 然后在每个状态选择 Q 值最大的动作。

2.1.2 部分可观测马尔可夫决策过程

强化学习中很多场景下环境都被建模为部分可观测马尔可夫决策过程 (Partially Observable Markov Decision Process, POMDP), POMDP 假设系统动态依旧是 MDP, 但执行动作的智能体不能获得 MDP 中的完整状态信息。POMDP 用七元组 $\langle \mathcal{S}, \mathcal{A}, \mathcal{O}, \mathcal{P}, \mathcal{Z}, \mathcal{R}, \gamma \rangle$ 表示。其中 $\mathcal{O}$ 为观测空间, $\mathcal{Z}(o | s', a)$ 为观测函数, 描述智能体执行动作 a 后到达状态 s' 时获得观测 o 的概率分布; 其余元素与 MDP 一致。结合深度学习, POMDP 的求解方法通常是使用记忆网络在观测历史序列 $h_t = (o_0, a_0, o_1, a_1, \dots, o_t)$ 的基础上做出决策, 缓解部分可观测带来的信息缺失问题。在本文所构建的《Tiny Swords》环境中, 智能体仅能观测到以自身为中心、有限半径内的其他智能体、怪物和奖励球, 超出视野范围的实体完全不可见, 且障碍物通过 36 条射线检测提供距离信息——这意味着每个智能体的观测仅是全局状态的局部投影, 天然构成 POMDP。

2.1.3 近端策略优化算法

近端策略优化算法 (PPO) 是本文主要使用的算法, 是一种策略梯度算法, 策略梯度算法通过直接优化策略函数 $\pi_\theta(a|s)$ 的参数 θ 来最大化期望累计回报 $J(\theta) = \mathbb{E}_{\pi_\theta}[G_t]$, 其梯度估计由策略梯度定理^[44]给出: $\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta}[\nabla_\theta \log \pi_\theta(a|s) A^\pi(s, a)]$, 其中 $A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s)$ 为优势函数。基于该思想提出的 AC 类算法如 A2C 等是单步估计优势函数, 这种方法导致早期估计不准时有较大的偏差, 而若使用多步估计优势函数, 则又会引入较大的方差。因此研究者通过广义优势估计 (Generalized Advantage Estimation, GAE)^[14]来代替单步估计优势函数, 通过 TD 误差的指数加权和在偏差与方差之间取得平衡:

$$\hat{A}_t^{\text{GAE}(\gamma, \lambda)} = \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l} \quad (2-5)$$

其中 $\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$ 为单步 TD 误差, $\lambda \in [0, 1]$ 控制偏差-方差权衡, $\lambda = 0$ 退化为单步 TD, $\lambda = 1$ 退化为 Monte Carlo。

置信域策略优化 (TRPO)。在使用广义优势估计时, 需要先通过上一步更新后的策略与环境交互收集一条轨迹, 再用轨迹更新策略, 为了提高样本利用率, 同一条轨迹可能会多次用来更新策略, 因此需要使用重要性采样来纠正多次更新后新策略和旧策略之间的分布偏移。但是这个偏移可能会比较大进而导致策略更新剧烈, 方差大, 不稳定。Schulman 等人^[45]提出的置信域策略优化 (Trust Region Policy Optimization, TRPO) 通过引入显式的策略更新约束来应对上述问题。TRPO 在最大化旧策略采样下的替代优势函

数的同时，要求新旧策略间的 KL 散度不超过预设阈值 δ ：

$$\begin{aligned} \max_{\theta} \quad & \mathbb{E}_t \left[\frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)} \hat{A}_t \right] \\ \text{s.t.} \quad & \mathbb{E}_t [D_{\text{KL}}(\pi_{\theta_{\text{old}}}(\cdot|s_t) \parallel \pi_{\theta}(\cdot|s_t))] \leq \delta \end{aligned} \quad (2-6)$$

其中重要性采样比率 $r_t(\theta) = \pi_{\theta}(a_t|s_t)/\pi_{\theta_{\text{old}}}(a_t|s_t)$ 用于纠正当数据来自 $\pi_{\theta_{\text{old}}}$ 而非 π_{θ} 时产生的分布偏移。TRPO 采用共轭梯度法结合线搜索近似求解该约束优化问题，在理论上保证了策略的单调改进^[45]。然而，其二阶优化的计算开销和工程实现复杂度限制了在大规模问题中的实用性。

近端策略优化。为在保留 TRPO 稳定更新特性的同时大幅降低实现与计算复杂度，近端策略优化（PPO）使用了两种方式：

一是PPO-Penalty——将 TRPO 中的硬 KL 约束转化为目标函数中的自适应惩罚项：

$$L^{\text{KL PEN}}(\theta) = \mathbb{E}_t \left[r_t(\theta) \hat{A}_t - \beta D_{\text{KL}}(\pi_{\theta_{\text{old}}}(\cdot|s_t) \parallel \pi_{\theta}(\cdot|s_t)) \right] \quad (2-7)$$

其中 β 为自适应惩罚系数。在每轮更新后，若实际 KL 散度超过目标阈值 d_{targ} 的 1.5 倍，则 $\beta \leftarrow 2\beta$ 以强化约束；若实际 KL 散度低于 $d_{\text{targ}}/1.5$ ，则 $\beta \leftarrow \beta/2$ 以放宽步长限制。这一机制使策略更新的幅度在训练过程中自动调节，无需手动设定硬性 KL 阈值。

二是PPO-Clip——采用了更加直接的裁剪策略，完全免除了 KL 散度的显式计算：

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right] \quad (2-8)$$

其中 ϵ 为裁剪超参数。PPO-Clip 通过简单的裁剪操作以极低的实现成本隐式约束了单步策略更新幅度。

在实际部署中，完整的 PPO 目标函数还需纳入价值函数误差和策略熵正则项：

$$L^{\text{PPO}}(\theta, \phi) = \mathbb{E}_t \left[L^{\text{CLIP}}(\theta) - c_1 (V_{\phi}(s_t) - V_t^{\text{targ}})^2 + c_2 S[\pi_{\theta}](\cdot|s_t) \right] \quad (2-9)$$

其中 V_t^{targ} 为目标价值，由GAE估计的优势进行构造， $S[\pi_{\theta}]$ 为策略熵， c_1, c_2 为各损失项的权重系数。

2.2 多智能体强化学习

MDP要求状态转移概率是恒定的，即环境是平稳的。对于多智能体系统，MDP被扩展为马尔可夫博弈（Markov Game） $\langle \mathcal{S}, \mathcal{N}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma \rangle$ ，其中 $\mathcal{N}$ 是智能体数量， $\mathcal{A} = A_1 \times A_2 \times \cdots \times A_N$ 为联合动作空间，状态转移和奖励均取决于所有智能体的联合动作： $\mathcal{P} : \mathcal{S} \times A_1 \times \cdots \times A_N \rightarrow [0, 1]$ ， $\mathcal{R} : \mathcal{S} \times A_1 \times \cdots \times A_N \rightarrow \mathbb{R}$ 。尽管转移函数 $\mathcal{P}$ 本身是平稳的，但从单个智能体 i 的视角看，其他智能体的策略 π_{-i} 构成了环境的一部分。当对手策略随学

习过程不断更新时，智能体 i 所感知的有效转移动态

$$P_i(s' | s, a_i) = \sum_{a_{-i}} \mathcal{P}(s' | s, a_i, a_{-i}) \pi_{-i}(a_{-i} | s) \quad (2-10)$$

也随之漂移，破坏了单智能体视角下马尔可夫性质。

2.3 奖励塑形理论

如1.2.4节所述，PBRS 通过在环境奖励上叠加形如 $F(s, a, s') = \gamma\Phi(s') - \Phi(s)$ 的塑形项来加速学习而不改变最优策略。其中 $\Phi: \mathcal{S} \rightarrow \mathbb{R}$ 为势函数，其设计依赖于领域知识——不恰当的势函数反而会拖慢训练^[46]。在本文中，势函数 $\Phi(s)$ 定义为智能体到目标奖励球的距离度量，使得 PBRS 提供朝向奖励球的稠密引导信号，具体势函数形式对比见第4.3节。

2.4 Godot 引擎与 RL 集成框架

Godot 是一款开源跨平台的 2D/3D 游戏引擎，提供完整的物理引擎、场景系统和脚本语言（GDScript），具有安装体积小、开发迭代快和授权限制少等特点，适合快速构建强化学习训练环境。除独立游戏原型外，Godot 也逐渐被用于更大规模的商业项目；例如《Slay the Spire 2》（《杀戮尖塔2》）采用 Godot 引擎开发，并于 2026 年进入 Early Access 阶段。这说明 Godot 在保持轻量化和开源优势的同时，也具备承载较复杂商业游戏项目的工程潜力。

Godot RL Agents 插件^[26]提供了一个标准化的强化学习训练接口，通过 AIController 节点将游戏逻辑与 RL 算法解耦，只需要实现其中几个关键方法就能像使用 Gymnasium 一样使用自定义的环境。其通信架构基于 TCP Socket：Godot 作为客户端，在每个物理帧将智能体的观测向量序列化发送给 Python 服务端，Python 端执行推理后将动作返回给 Godot，实现了游戏逻辑（GDScript）与训练逻辑（Python/PyTorch）的完全分离。在环境接口层面，本系统采用 Godot RL Agents 提供的基于 PettingZoo 的 Parallel API 规范封装环境，支持并行运行多个环境加速采样，实现分布式训练。图 2-2 展示了整个系统架构。

第3章 博弈环境设计与实现

3.1 游戏总体设计

《Tiny Swords》是一款基于 Godot 引擎的 2D 俯视角四人混战竞技场游戏，旨在作为多智能体强化学习的博弈环境。四名角色（Agent）在一个封闭的竞技场中进行自由对

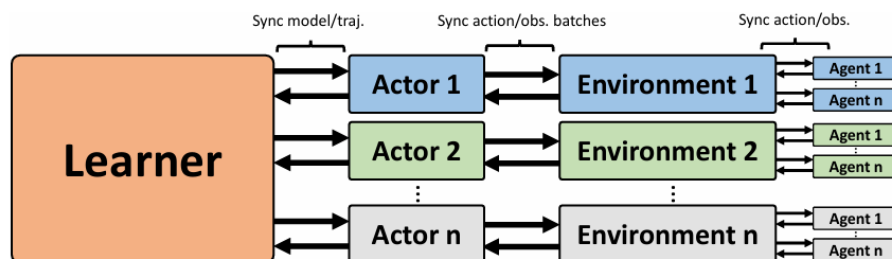


图 2-2: 系统架构：分布式训练环境，Python端的强化学习模块为Learner，负责梯度计算与参数更新；策略网络为Actor，负责发出动作指令与环境交互获取观测；Godot端的环境模块为Environment，负责动作执行模拟，返回新的观测，奖励和终止信号等；Godot游戏中的角色为Agent，负责执行游戏中实际的动作

抗，通过移动与近战攻击争夺分数。游戏在核心对抗之上叠加了少量环境交互元素（野生怪物、分数球），以丰富策略空间并产生更具层次的博弈行为。

游戏包含以下核心元素：

- **竞技场：**竞技场为一个固定大小的矩形封闭地图，四周由不可通行的墙壁围成。四名智能体分别在地图的四个角落出生。竞技场内有不可通行的障碍物，增加对抗复杂性和路径规划难度。
- **智能体：**拥有生命值系统，可执行移动（上下左右）、待机和攻击共 6 种离散动作，死亡后在各自出生点重生。
- **野怪（敌人）：**由内置程序控制的NPC，在竞技场中心区域内随机游走，附近出现智能体在一定范围内追杀智能体，被击杀后掉落分数并在一段时间后重新刷新。
- **奖励球：**智能体拾取到可以获得奖励，分 A 类和 B 类两种，A类球出生在智能体的出生点附近，被拾取后不再重生，B类球出生在地图中央区域，被收集后经过冷却时间重新生成。
- **中央资源密集区域：**为了增加智能体照面机会，将竞技场一定的中心区域设置为资源密集区，B类球只在该区域内重生，敌人只在该区域内巡逻。
- **视野限制：**智能体仅能感知视野半径内的实体，模拟部分可观测环境。

每局游戏持续3分钟，4 名智能体同时行动，每8个物理帧进行一次观测并执行一次动作，Godot引擎每秒会处理60个物理帧，因此总共有10800个物理帧，智能体在每局游戏会执行1350次动作。表 3-1 给出了本文环境实现中的更多细节。

3.2 观测空间设计

观测空间的设计直接影响策略网络和价值网络对游戏状态的理解能力。本文设计了174 维分段式观测向量，将异构的游戏信息按语义分组编码：

关键设计特点：

表 3-1: 游戏环境主要参数

类别	参数	取值与说明
地图	尺寸	18 × 18 栅格正方形地图，地图整体按 1.5 倍缩放，栅格边长为 96 像素
观测	视野半径	200 像素
智能体	生命值与速度	生命值 100，移动速度 100 像素/秒
智能体	攻击参数	攻击力 15，攻击范围为正前方半径 18 像素、圆心角 100° 的扇形，攻击冷却时间 1.5 秒
敌人	数量与属性	最多同时存在 5 个；每个敌人生命值 30，攻击力 10，攻击冷却 2.5 秒，攻击范围与智能体相同，被击杀后 7 秒重生
奖励球	A 类球	每名智能体出生点附近开局生成 3 个，被拾取后不再重生
奖励球	B 类球	最多同时存在 5 个，出生于地图中央区域，被拾取后 6 秒刷新
回合	时间与决策间隔	每局 3 分钟；每 8 个物理帧执行一次动作，共 1350 个决策步

表 3-2: 观测空间组成

分段	维度	实体数 × 实体维度	内容
自身状态 (SELF)	15	1 × 15	坐标、血量、速度、上一动作 one-hot 等
其他智能体 (PLAYER)	33	3 × 11	相对位置、血量、距离、攻击状态等
奖励球 (BALL)	40	8 × 5	相对位置、类型、距离等
敌人 (ENEMY)	50	5 × 10	相对位置、血量、攻击状态等
地图射线 (LiDAR)	36	36 × 1	36 条射线检测距离，归一化到 [0,1]

- (1) **以自身为中心**：所有坐标转换为相对于智能体的偏移量，除以视野半径归一化，确保平移不变性。
- (2) **固定槽位编码**：智能体按 ID 固定槽位、敌人按名称排序、奖励球按距离排序，固定槽位的设定使得在每个时间步上，同一槽位始终承载同一类语义信息，直接将原始特征送入 GRU 即可保持跨时间步的隐藏状态位置一致性，因此无需前置 MLP 投影层。
- (3) **有效位掩码**：在视野外的实体属于无效信息，将采用 0 值填充，每个实体槽位后附 1 维有效掩码（valid mask），使网络明确区分有效信息与填充零值。
- (4) **LiDAR 射线检测**：从智能体位置向四周均匀发射 36 条射线，记录与障碍物碰撞点的距离并归一化，提供简洁高效的局部地图感知。

3.3 奖励函数设计

奖励函数是智能体行为的主要驱动力。本文从事件稀疏奖励和稠密塑形奖励两个层次构建奖励体系。

事件奖励定义游戏关键事件的奖罚值。基础奖励体系包含以下事件类别：拾取得分（A 球/B 球）、撞墙、击杀（敌人/其他智能体）、死亡惩罚，此外，也有比较稠密的事件：造成伤害（对敌人/对其他智能体）、受到伤害、攻击惩罚、待机惩罚、移动惩罚，同时为了鼓励智能体在中心区域，相应的也有中心区域稠密奖励。

为了观察到风格不同的智能体，引入了 **per-player 差异化**：为 4 名智能体配置不同的奖励以诱导多样化的策略风格。表 3-3 列出了各智能体的完整奖励配置。在第 4 章对比实验中，除网络架构对比实验采用单智能体环境并保留差异化奖励外，其余多智能体环境下的消融实验均使用同一套奖励配置，以减少不同奖励偏好对方案比较的干扰。智能体 1 提升吃球奖励并降低战斗收益，诱导避战采集策略；智能体 0 和智能体 2 则相反，偏好战斗导向；智能体 3 采用均衡但谨慎的风格——战斗奖励低于战士型智能体、高于采集型智能体，承受的负面惩罚也较重，整体倾向于以低风险方式积累分数。

奖励球势能塑形基于 PBRs 理论设计，目标是将稀疏的吃球奖励转化为稠密的引导信号。本文以智能体到奖励球的距离作为势函数输入，并将线性势函数作为候选方案之一：

$$\Phi_{\text{ball}}(s) = -\frac{R_{\text{ball}}}{r_{\text{vision}}} \cdot d + R_{\text{ball}} \quad (3-1)$$

其中 R_{ball} 为球的分值， r_{vision} 为视野半径， d 为智能体到球的距离。由于环境中同时存在多个奖励球，本文进一步比较仅对视野内最近奖励球计算势能和对所有可见奖励球叠加势能两种方式，具体效果在第 4.3 节中分析。

障碍物避障围绕稀疏碰撞惩罚和稠密避障塑形两类方案展开。稀疏方案仅在智能体与墙体或障碍物发生碰撞时给予惩罚；稠密方案则根据智能体到障碍物的距离构造势函数或直接距离惩罚。第 4.4 节将对不同惩罚强度与塑形方式进行对比。

表 3-3: 各智能体奖励配置

奖励事件	P0 (战士型)	P1 (采集型)	P2 (战士型)	P3 (均衡型)
拾取 A 球	6	12	6	6
拾取 B 球	3	10	3	4
对敌人造成伤害	8	3	8	4.05
对其他智能体造成伤害	10	4	10	6.05
击杀敌人	16	6	16	8
击杀其他智能体	20	8	20	12
受到伤害	-5	-10	-5	-8
死亡	-10	-20	-10	-15
攻击动作 (/次)	-0.01	-0.06	-0.01	-0.05
移动 (/帧)	-0.001	-0.001	-0.001	-0.001
静止 (/帧) *	-0.005	-0.005	-0.005	-0.005
中心区域奖励 (/帧)	0.003	0.003	0.003	0.003
撞墙惩罚	-0.5	-0.5	-0.5	-0.5

* 静止（待机）惩罚在中心区域内豁免：当智能体位于竞技场中心区域（patrol_rect）时，不施加待机惩罚，同时每帧获得中心区域正奖励。

奖励归一化：采用 Welford 在线算法维护奖励的 running mean/variance，对奖励做 z-score 归一化，稳定价值网络的输出在[-1,1]之间，以减小拟合难度。

3.4 课程学习环境

为解决智能体环境训练难度高、学习信号稀疏的问题，本文设计了 5 阶段课程学习（Curriculum Learning）方案，从简单到复杂逐步引入游戏元素，并行训练4个智能体，得到4个表现较好的模型：

阶段1：无障碍拾取奖励球：无墙、无敌人，仅包含奖励球。智能体学习基础的移动和吃球行为。

阶段2：有障碍物拾取奖励球：引入障碍物，智能体需要在避障的同时收集奖励球。

阶段3：敌人战斗：将智能体围在中心的小区域内，仅包含敌人，无奖励球。智能体被迫学习基础的战斗技巧。

阶段4：敌人和障碍物：在上一课程基础上加入障碍物和奖励球，但智能体还是被围在中心区域内，无需主动前往中央资源密集区。

阶段5：完整环境：包含所有游戏元素，智能体综合运用前述技能。

每个阶段以前一阶段的最优模型作为初始化权重，使智能体在保持已习得技能的基础上逐步适应更复杂的环境。课程学习有效缓解了从头开始在完整环境中训练的收敛困难。

第4章 单智能体基础训练与对比实验

在进入多智能体博弈之前，首先需要确保单智能体具备完成基础游戏任务的能力。本章通过系统的对比实验，筛选适用于本文环境的配置与训练策略。

4.1 训练框架

为了能有对算法良好的控制性，训练框架基于 CleanRL 风格的手写 PPO 实现，针对多智能体场景扩展为 IPPO：每个智能体拥有独立的 Actor 网络（策略网络）和 Critic 网络（价值网络），使用共享的 rollout buffer 同步收集交互数据，各自独立更新网络参数。网络架构支持三种模式：

- **MLP**：将所有 174 维观测拼接后输入多层全连接网络。
- **Segmented MLP**：5 个独立 MLP 分别编码 SELF/PLAYER/BALL/ENEMY/LiDAR 分段，分别由一个 MLP 子网络处理，子网络的输出拼接后进入共享 MLP trunk。
- **GRU-MLP**：时序相关分段通过 GRU 循环层处理，奖励球分段送入 MLP，在输出后与 GRU 网络输出拼接后进入 MLP trunk。

图 4-1 展示了三种网络架构的详细结构。

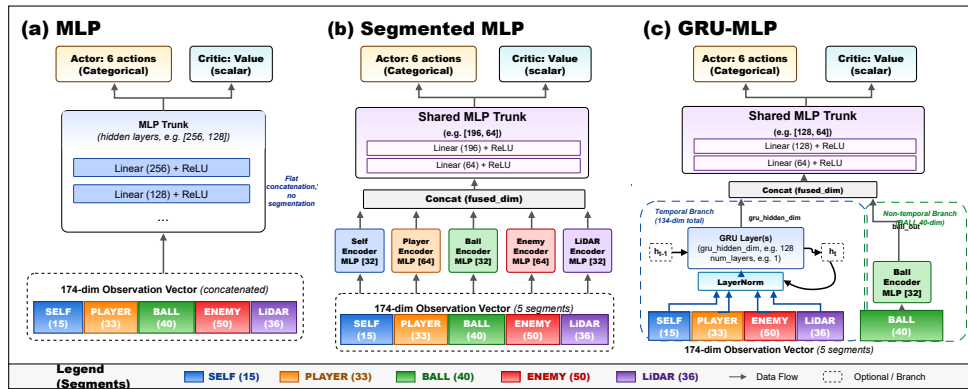


图 4-1: 三种网络架构详细结构对比。(a) MLP 将全部 174 维观测拼接后经多层全连接网络处理；(b) Segmented MLP 通过 5 个独立 MLP 分别编码 SELF/PLAYER/BALL/ENEMY/LiDAR 分段，输出拼接后进入共享 MLP trunk；(c) GRU-MLP 将时序相关分段（SELF、PLAYER、ENEMY、LiDAR 共 134 维）送入 GRU 循环层建模时序依赖，非时序的 BALL 分段经独立 MLP 编码后与 GRU 输出拼接，共同进入共享 MLP trunk。

除特别说明外，本章所有对比实验均从随机初始化的策略网络和价值网络开始训练，用于考察不同设计方案对训练效率的影响。多智能体环境实验使用统一奖励配置，网络架构对比实验使用单智能体环境并保留表 3-3 中的差异化奖励设置。每组实验均从随机初始

化开始进行一次训练；训练时同时运行多个并行环境，通常为 5 个，部分实验为 10 个。并行环境之间采用连续随机种子，即第一个环境使用随机种子 0，其余环境种子依次加 1。所有训练曲线均按横轴总长度的 5% 作为窗口进行滑动平均，以减小随机波动对趋势判断的影响。

4.2 观测空间消融：有效掩码

有效掩码（valid mask）用于标记观测向量中哪些槽位包含有效信息，消除因视野内实体数量不足而产生的零填充噪声。图 4-2 展示了是否启用有效掩码对智能体表现的影响：a 为所有智能体总吃球得分均值，b 为各智能体单独吃球得分均值。

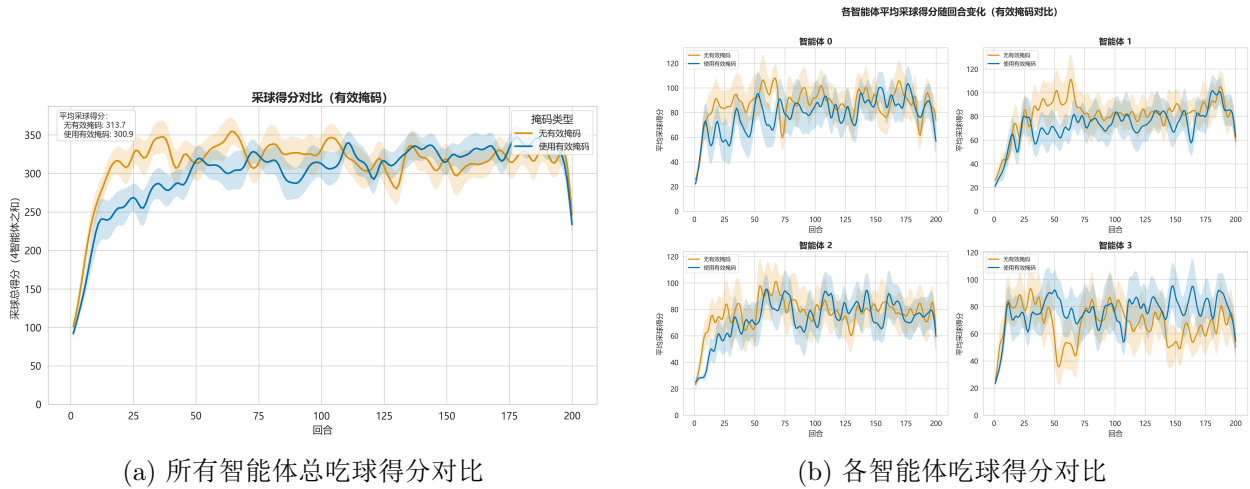


图 4-2: 有效掩码消融实验结果。蓝色为未启用有效掩码，橙色为启用有效掩码。曲线为 5 次独立运行均值。

实验结果表明：

- 启用有效掩码后全场总得分提升 9.9%，表明区分有效与无效观测信息有助于智能体做出更优决策；详细数据见附录表 A-1。
- 训练初期启用掩码的得分略低，因每个实体槽位额外增加 1 维有效位，观测维度由 158 维增加至 174 维，但约在 50 回合后开始反超，说明网络逐渐学会了利用 valid 位筛选有效信息。
- 智能体 3（出生点附近有障碍物，历史表现最弱）改善幅度最大，有效掩码对弱势智能体的帮助尤为突出。
- 为观测数据的真实性和训练稳定性，后续实验均启用有效掩码。

4.3 奖励球塑形方案对比

正如前文所述，不同的势函数对训练效率有显著影响，而如何设计合适的势函数是需要专业领域知识的，本文所构建的游戏环境不是被大量研究过的领域，不存在所谓的专

家，因此每种方案都需要进行系统实验和对比。对于奖励球这类空间目标导航类任务，常见的奖励塑形方法有基于势能函数的塑形，距离奖励（每次靠近给予奖励，远离给予惩罚）。不同的奖励塑形方案直接影响智能体吃球行为的学习效率。

针对势函数，本文设计了三种候选势函数进行对比：

$$\Phi_{\text{linear}}(s) = -\frac{R_{\text{ball}}}{r_{\text{vision}}} \cdot d + R_{\text{ball}} \quad (4-1)$$

$$\Phi_{\text{exp}}(s) = R_{\text{ball}} \cdot e^{-d/r_{\text{vision}}} \quad (4-2)$$

$$\Phi_{\text{inverse}}(s) = R_{\text{ball}} \cdot \frac{r_{\text{vision}}}{d + r_{\text{vision}}} \quad (4-3)$$

在 500,000 时间步下，不同奖励塑形方案对拾取奖励球得分的影响如图 4-3所示。可以发现距离奖励，线性势函数，反比例势函数3个方案没有明显差距；而指数势函数则表现居中，这可能是因为指数函数变化剧烈，在离奖励球比较远时引导程度不如其他势函数大，导致在较远距离依旧呈现稀疏性；稀疏奖励效果最差，这是意料之中的。

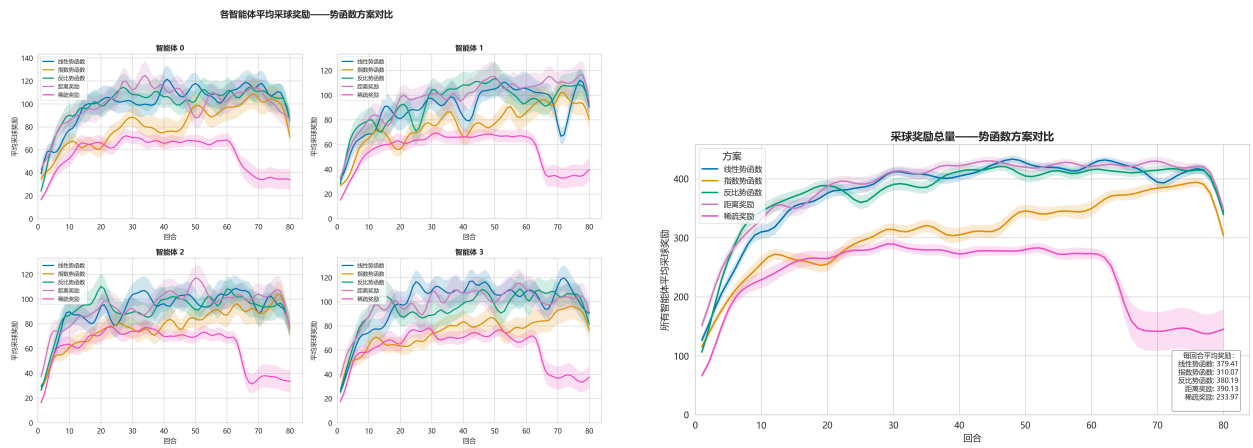


图 4-3: 不同奖励塑形方案对拾取奖励球得分的影响。a为各智能体得分，b为全场平均得分变化趋势。

对于本文特定环境，智能体需要拾取多个奖励球，通常的奖励塑形方法都用于单目标引导，对于多目标引导，在采用线性势函数的情况下，还对比了仅对最近球计算势能和视野内所有球计算势能两种模式，即考虑是否要将多目标任务划分为多阶段的子目标，结果如图 4-4所示，最近球模式的训练效率高出 67%，表明将多目标任务划分为多阶段子目标在本文环境中能够有效提高训练效率。推测原因是多球势能叠加在球之间的中点区域会相互抵消，形成势能平台区，削弱了引导效果，并且理论上在使用线性势函数时，该现象更明显，实验结果也符合这一点。

4.4 障碍物避障策略

障碍物的存在增加了移动规划的难度。本文从惩罚力度和塑形方式两个维度进行系统实验。

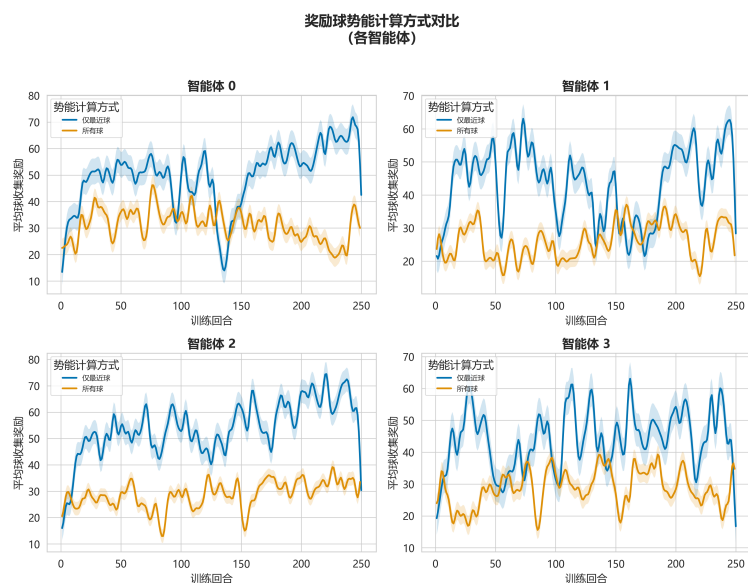


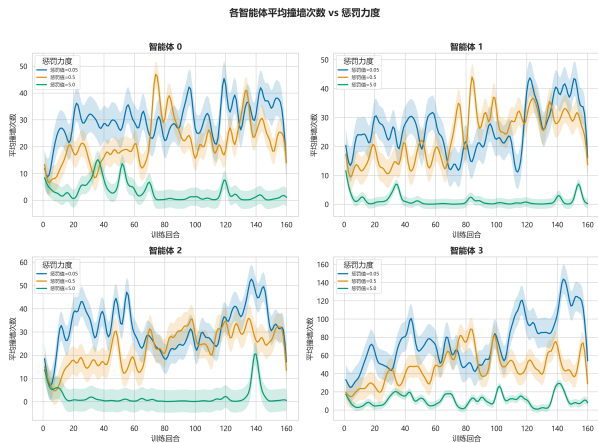
图 4-4: 各智能体奖励球势能计算方式与回合奖励的关系

撞墙惩罚力度对比：在稀疏惩罚（仅碰撞时惩罚）策略下，分别测试了 0.05、0.5 和 5.0 三种惩罚值。图 4-5展示了三种惩罚力度下智能体的撞墙行为与吃球收益随训练回合数的变化趋势：a 为各智能体单独撞墙次数，b 为全场平均撞墙次数，c 为各智能体单独吃球得分，d 为全场总吃球得分。

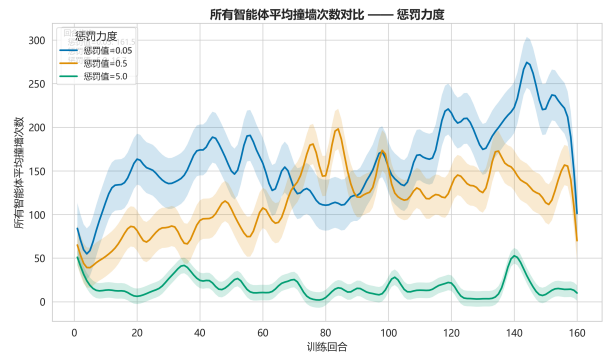
实验结果表明，0.5 惩罚在得分与避障之间取得了较好的平衡：吃球得分与 0.05 基本持平（274.1 vs 272.6），撞墙次数减少 29%（114.8 vs 161.5）。5.0 惩罚虽将撞墙次数降至 16.7，但吃球得分大幅下降至 122.0，说明过强的惩罚导致智能体为避免墙壁惩罚而过度保守，牺牲了大量靠近墙壁的奖励球收益。进一步观察各智能体子图可以发现，不同智能体在相同惩罚力度下的学习曲线存在明显差异：智能体 3 在低惩罚（0.05）下的撞墙次数最高（75次/回合），而智能体 2 在惩罚 0.5 下的吃球得分最高（65.3），反映出不同出生位置和局部障碍物分布对墙壁接触概率的结构性影响。

障碍物塑形方案对比：以 0.5 稀疏惩罚为基线，对比了线性 PBRS、反比例 PBRS 和距离直接惩罚三种方案。图 4-6展示了四种方案下智能体的行为对比：a 和 b 分别为各智能体及全场平均撞墙次数，c 和 d 分别为各智能体及全场总得分。

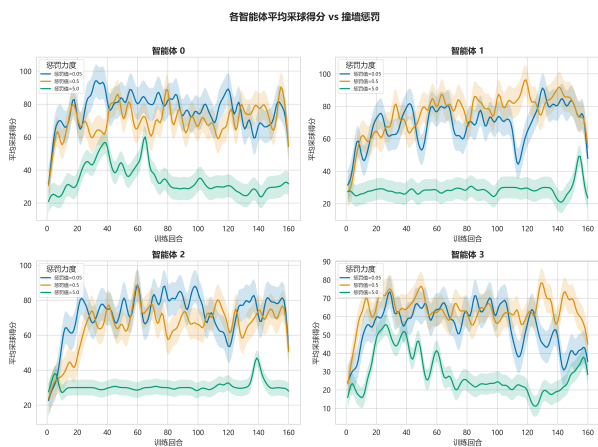
实验发现三种塑形方案均明显减少了撞墙次数：线性 PBRS 减少 54.2%（114.8 → 52.6），反比例 PBRS 减少 52.3%（114.8 → 54.7），距离直接惩罚减少 61.1%（114.8 → 44.7）。但稠密塑形也导致总得分略有下降（稀疏 209.5 vs 线性 203.4 / 反比例 199.3 / 距离 185.2），这主要是因为当对障碍物施加势能奖励时，远离障碍物可以获得奖励，但如果奖励球本身离障碍物很近，智能体同时受到奖励球和障碍物的双向势能影响，导致奖励球势能的引导性被削弱，对于障碍物附近的奖励球，智能体会倾向于忽略。障碍物与奖励球本质不同：障碍物并非“越远越好”，只要未发生实际碰撞就不应施加惩罚，综合权衡避障效果与得分损失，最终采用稀疏惩罚策略。



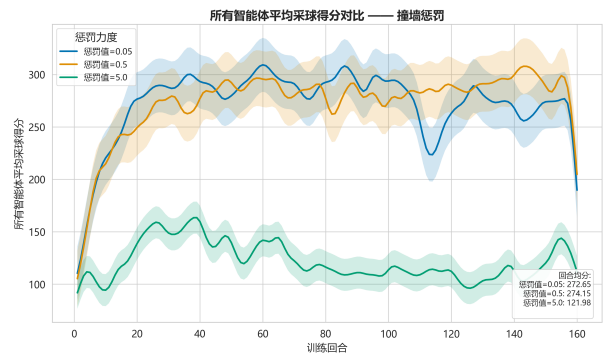
(a) 各智能体撞墙次数对比



(b) 所有智能体平均撞墙次数对比



(c) 各智能体吃球得分对比



(d) 所有智能体总吃球得分对比

图 4-5: 不同撞墙惩罚力度 (0.05、0.5、5.0) 下智能体行为对比。蓝色为 $\text{Penalty}=0.05$, 橙色为 $\text{Penalty}=0.5$, 绿色为 $\text{Penalty}=5.0$ 。曲线为 5 次独立运行均值。横轴为训练回合数 (Episode), 纵轴分别为撞墙次数 (次/回合) 和吃球奖励得分。

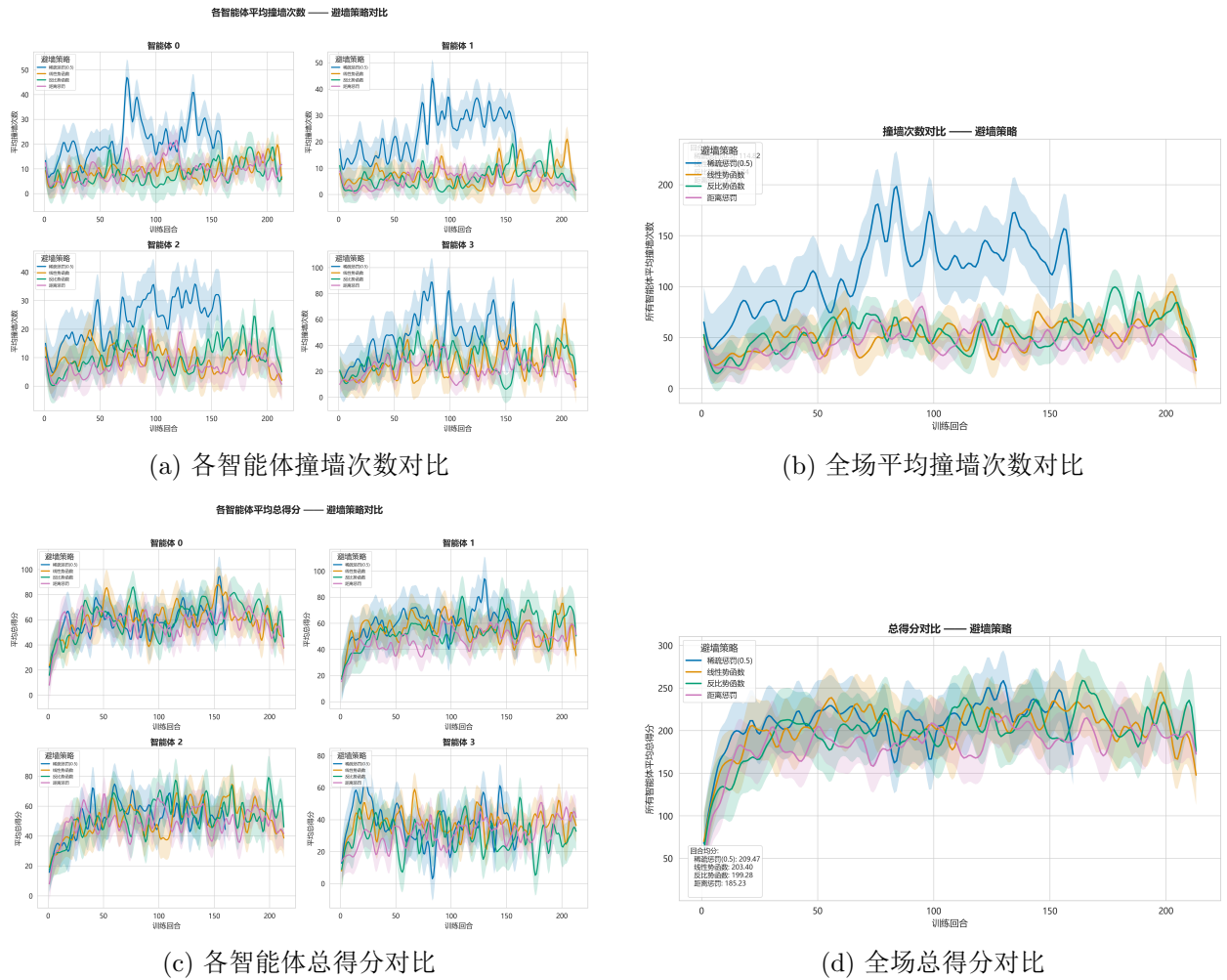


图 4-6: 不同障碍物塑形方案下智能体行为对比。蓝色为 Sparse Penalty (基线 0.5), 橙色为 Linear Potential, 绿色为 Inverse Proportional Potential, 粉色为 Distance Penalty。曲线为 5 次独立运行均值。

4.5 网络架构对比

对于部分可观测马尔可夫决策过程，时序网络的表现通常优于MLP等非时序网络^[47]。在单智能体环境下，对比了 MLP、Segmented MLP 和 GRU-MLP 三种网络架构，其中GRU网络还对比了96和128两种隐藏层维度，MLP、Segmented MLP 和 GRU-96-MLP保持总参数量大体相当。图 4-7展示了四种网络架构在两种任务场景下的表现对比：a为拾取奖励球任务，即使用课程1，场景中只有奖励球，没有障碍物和敌人，b为单智能体战斗任务，即使用课程3，智能体和敌人被围在一个小的中心区域中，智能体将被迫战斗。

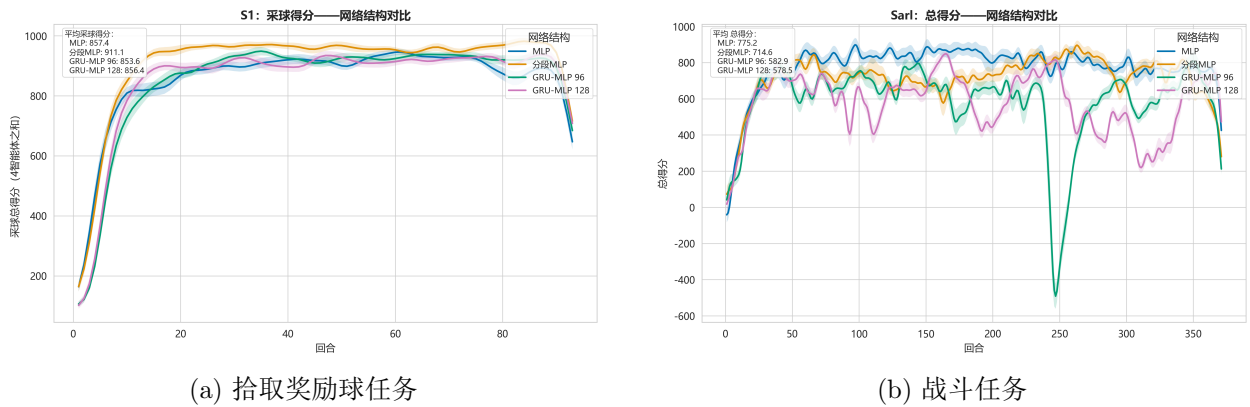


图 4-7: 不同网络架构在两种任务场景下的总得分对比。蓝色为 MLP，橙色为 Segmented MLP，绿色为 GRU-MLP（隐藏层 96），粉色为 GRU-MLP（隐藏层 128）。曲线为 5 次独立运行均值。

拾取奖励球任务：Segmented MLP 表现较好（均值 911.1），略高于 MLP（857.4）和 GRU-MLP（853.6–856.4）。在仅需感知奖励球位置的简单任务中，Segmented MLP 对观测槽位的分段编码略有益处，因为分段子网络的设计使其在这个简单任务里不会受到其他大量0值输入的干扰。但四种架构在学习趋势上未呈现质变差异，表明该任务复杂度尚不足以充分体现网络结构的优劣。

战斗任务：普通 MLP 在总分上表现较好（775.2），Segmented MLP 次之（714.6），GRU-MLP 总分明显落后（582.9 / 578.5）。值得注意的是 GRU-MLP 128 的攻击次数最高（79.0 vs MLP 的 69.5），且受击次数和死亡次数略优，但其撞墙次数极高（GRU-MLP 96: 23.6 次/回合 vs MLP: 3.4 次/回合），反映出时序网络在该环境中对空间避障信息的利用效率不足。在攻击密集的实时战斗场景中，GRU 的时序依赖带来的学习成本反而拖累了训练效率。需要说明的是，受实验周期限制，本文未对 GRU 的序列长度、隐藏层规模、学习率等超参数进行单独搜索，因此上述结果只能说明在当前训练预算和参数设置下 MLP 更适合作为后续实验的基础架构，而不能排除经过充分调参后的循环网络取得更好表现的可能。综合训练成本与当前实验结果，后续实验选择 MLP 作为主训练网络架构。各网络在战斗任务中的详细指标对比见附录表A-6。

上述实验结果与直觉预期相反——在部分可观测环境中，时序网络通常应优于 MLP。经分析，这一现象主要源于本实验环境设计的两个特征。其一，如图 4-8所示，竞技场中央土色区域为资源密集区，随着训练进行，智能体会倾向于前往该区域并停留以试图获取

更高奖励，在智能体的活动高度集中于该区域的情况下，智能体的视野半径设置较大，使得处于中央的智能体的视野可覆盖竞技场核心区域的大部分信息，大部分时间内智能体总能看到其他实体，MLP 的单帧观测已包含充分的决策依据。其二，观测向量中直接编码了对手的攻击状态（如是否在播放攻击动画以及攻击冷却时间）与移动速度等动态信息，网络无需依赖历史记忆来推断“上一秒发动攻击，现在无法攻击”这类时序关联关系。因此在本环境和当前训练设置下，GRU 的时序建模能力未能转化为实际优势，反而增加了训练收敛难度；后续实验采用 MLP 作为训练网络架构。

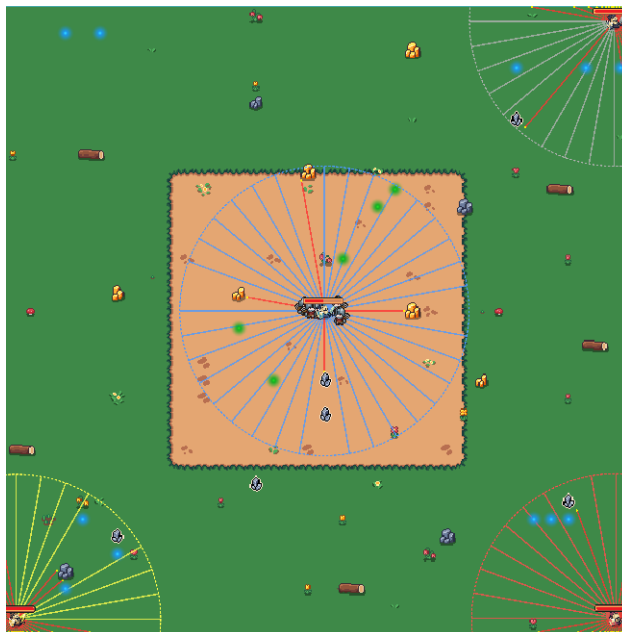


图 4-8: 竞技场俯视图。土色矩形区域为资源密集区，虚线圆为智能体视野范围示意。智能体位于中央位置时可观测到核心区域的大部分动态信息。

超参数配置：确定使用MLP作为训练网络架构后，在最终环境下经过 Optuna 参数搜索得到了一组效果比较好的超参数配置。搜索过程在单智能体环境下以智能体 0 为训练对象，共进行 100 个 trial，优化目标为最近 100 回合的平均奖励，搜索得到的最优目标值为 830.45。主要搜索空间如表 4-1所示，最终采用的关键超参数由表 4-2 给出，图 4-9展示了超参数搜索结果。此后各课程会在此基础上动态调整以取得较好性能——例如战斗课程需要更大的熵系数以鼓励探索。

第5章 多智能体博弈方法探索

邓小铁等人已经证明：在多智能体博弈中，试图同时提高每个智能体策略的纳什均衡求解是 PPAD-完全的^[48]，独立策略梯度方法在两人零和博弈中才有机会收敛到纳什均衡^[49]。本文的环境是一个多人不对称对抗环境，且每个智能体拥有不同的奖励配置，属于多人一般和博弈，本文将不试图直接尝试求解多人一般和博弈，而是以其中一个智能体为代表，分别测试IPPO，简化的联盟训练，以及虚拟博弈三种方法哪种最能够在博

表 4-1: Optuna 超参数搜索空间

参数	搜索范围或候选值
学习率	$[1 \times 10^{-5}, 1 \times 10^{-3}]$ ，对数均匀采样
Rollout 步数	{64, 128, 256}
PPO 裁剪系数 ϵ	[0.1, 0.3]，均匀采样
熵系数	$[1 \times 10^{-4}, 5 \times 10^{-2}]$ ，对数均匀采样
学习率退火	{False, True}
奖励归一化	{True, False}
奖励归一化范围	启用奖励归一化时，从 {1.0, 2.0, 3.0, 4.0, 5.0} 中选择
MLP 隐藏层结构	{128-64, 256-128-64, 256-256-128}

表 4-2: PPO 训练超参数

参数	取值
学习率	7.22×10^{-4}
折扣因子 γ	0.99
GAE 参数 λ	0.95
Rollout 步数	256
Minibatch 数	4
更新轮数	5
PPO 裁剪系数 ϵ	0.2
熵系数	0.01
梯度裁剪	4.0
学习率退火	True
奖励归一化	True
奖励归一化范围	[-1.0, 1.0]

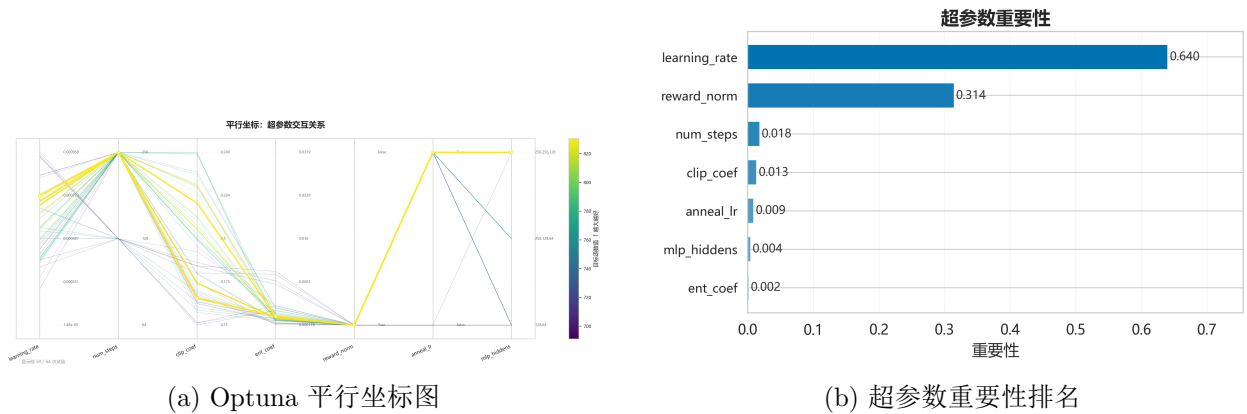


图 4-9: Optuna 超参数搜索结果可视化。a展示各 trial 的超参数取值与对应奖励的分布关系，颜色越接近金色表示奖励越高；b展示各参数对最终奖励的影响权重。

弈训练过程中让智能体学到更加鲁棒的策略，能应对多种风格不一的对手。测试分为三步：（1）在通过课程学习训练的四个智能体模型的基础上，将其同时放置于环境中，使用IPPO同步训练一段时间，让每个智能体能够感知到其他智能体。（2）采用轮换训练初始化对手池，由于环境是不对称博弈，因此智能体与自身的历史版本对抗分数会较高，没有对抗意义，因此对手池中不包括自身策略。（3）分别用IPPO，简化联盟训练（只使用一个对手池，且其中不包含自身策略），虚拟博弈三种方法对智能体0进行训练，得到三个模型。（4）将三个模型分别和对手池中的策略进行对抗，分析其表现。测试流程如图 5-1所示。

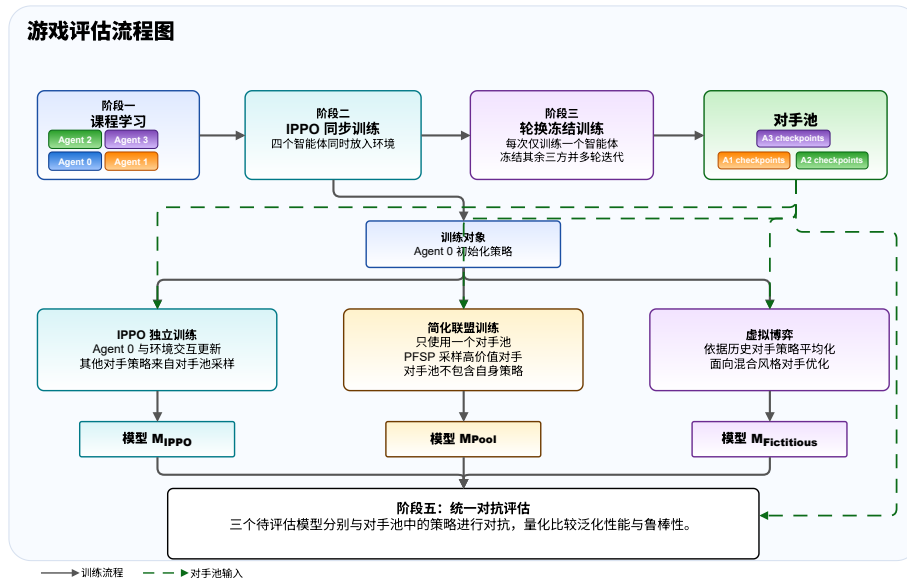


图 5-1: 评估流程。阶段一：通过五阶段课程学习训练四个智能体的基础模型。阶段二：将四个智能体同时放入环境进行 IPPO 同步训练，使各智能体感知对手存在。阶段三：采用轮换冻结方式，每次仅训练一个智能体且冻结其余三方，多轮迭代后收集最近检查点构建对手池。阶段四：分别使用 IPPO 独立训练、简化联盟训练（PFSP 采样对手池）和虚拟博弈（对手策略平均化）三种方法训练智能体 0，得到三个待评估模型。阶段五：将三个模型分别与对手池中的策略进行对抗，通过胜率矩阵、历史对战表现评分和行为分析量化对比三种训练方法的泛化性能。

5.1 对手池初始化

本文设计了迭代博弈（Iterative Play）训练框架用于初始化对手池。具体训练流程如下：

- 第1轮： **同步初始训练**：以通过课程学习的4个智能体为基础，4 个智能体同时进行 IPPO 训练 T_0 步，各自获得初始策略 $\pi_0^{(0)}, \pi_1^{(0)}, \pi_2^{(0)}, \pi_3^{(0)}$ 。
- 第2轮： **冻结对手**：从智能体 2、3、4 的检查点中选择策略并冻结，仅训练智能体 0 的策略 T_1 步，期间每隔一定步数保存检查点，获得更新后的策略 $\pi_0^{(1)}$ 。
- 第3轮： **轮换训练**：依次解冻下一个智能体、冻结其余三方，重复上述过程，获得

$$\pi_1^{(1)}, \pi_2^{(1)}, \pi_3^{(1)}。$$

第4轮： **多轮迭代**：重复步骤 2-3 共 K 轮，每轮各智能体独立进化 T_k 步。最后选取每个智能体最近20个检查点，作为对手池。

在该初始化方法下，每轮训练中被训练的智能体面对的是稳定的对手策略分布，绕过了非平稳的 MARL 问题同时让每个智能体在不同时间有效地面对了不同的敌人，期间保存的检查点的策略也就各不相同，风格各异，适合作为对手池。

5.2 待评估智能体的训练

IPPO：从对手池初始化过程的初始策略 $\pi_0^{(0)}, \pi_1^{(0)}, \pi_2^{(0)}, \pi_3^{(0)}$ 开始，使用IPPO训练 T_e 步，保存智能体0的策略 π_0^{Ippo} 。

简化联盟训练：对于智能体0，从对手池初始化过程的初始策略 $\pi_0^{(0)}$ 开始，每次从对手池中选择一个策略作为对手，进行对抗训练，训练 T_p 步，然后再从对手池中选择一个对手，重复若干次，保证训练步数为 T_e ，保存智能体0的策略 π_0^{Pool} 。每次选择对手时，采用PFSP（Prioritized Fictitious Self-Play）方法，从对手池中采样对手组合：

- 以概率 ε 均匀随机采样，保持对各类对手策略的统计探索。
- 以概率 $1 - \varepsilon$ 按历史奖励加权采样——主智能体对某对手组合的历史得分越低，该组合被选中的概率越高，使训练聚焦于当前最难应对的对手。

对手难度通过指数移动平均（EMA）奖励追踪：每轮对战结束后，维护主智能体对各对手组合 c 的 EMA 奖励 R_{ema}^c ，采样概率遵循：

$$P(c) \propto \exp\left(-\frac{R_{ema}^c - \min_c(R_{ema})}{T}\right) \quad (5-1)$$

其中 T 为温度参数，控制采样分布的集中程度； T 越小，对战后自身奖励越低的对手的采样权重越大，训练越聚焦于困难对手。

虚拟博弈：让智能体0的初始策略 $\pi_0^{(0)}$ 与对手池中的平均策略进行对抗训练，每次对手执行动作时，对手池中所有对手都会用各自的策略网络推理动作概率，然后将这些概率取平均作为最终动作概率，选取概率最大的动作执行，训练 T_e 步，保存智能体0的策略 $\pi_0^{Fictitious}$ 。

5.3 实验结果与讨论

（此处放置迭代自博弈的实验结果——胜率矩阵表、评分演化曲线图、行为分化对比图。）

实验发现的核心结论：

策略循环示意图（胜率矩阵热力图）
（预期结果：A 克 B，B 克 C，C 克 A）

图 5-2: 策略循环示意

评分随迭代轮次演化曲线占位图

图 5-3: 评分演化

- (1) 经过 K 轮迭代自博弈后，智能体的策略展现出显著的策略循环特征，胜率矩阵呈现非传递性模式。
- (2) 各智能体的评分在多轮迭代中交替上升，而非收敛到平稳值，验证了持续的对抗性进化。
- (3) 行为分析揭示了好战型与避战型策略的分化，且同一智能体在不同对手面前展现出行为适应性调整。
- (4) 相较于独立 IPPO 训练的基线（各智能体不进行迭代自博弈），迭代自博弈显著提升了智能体面对陌生对手时的泛化性能。

第6章 总结与展望

6.1 工作总结

本文围绕智能体博弈对抗这一主题，从环境构建、基础训练到博弈对抗三个层面开展了系统研究，主要工作总结如下：

（1）基于 Godot 引擎设计并实现了《Tiny Swords》2D 多智能体竞技场博弈环境，构建了 174 维分段式观测空间与 LiDAR 射线检测系统，为 MARL 研究提供了可控、可扩展的实验平台。

（2）基于 PBRS 理论设计了多层奖励塑形系统，包括奖励球线性势能引导、障碍物避障与 per-player 差异化事件奖励。通过大量消融实验，确定了线性势函数 + 最近球模式 + 0.5 稀疏撞墙惩罚的最优奖励配置。

（3）在课程学习框架下对比了 MLP、Segmented MLP 与 GRU-MLP 三种网络架构，实验表明 MLP 在战斗任务中以总分 775.2 的最优表现成为最合适的基础架构。

（4）构建了具有不同策略的对手池，测试对比了 IPPO、简化联盟训练和虚拟博弈三种方法在训练更具鲁棒性、能适应多种风格对手的智能体策略的训练效率和表现差异。

6.2 未来展望

本课题在以下方面仍有改进空间：

（1）**博弈模块的深入探讨：**由于将游戏设计成了纯对抗型的混战非对称对抗环境，主流用于智能体博弈训练的方法不完全适用，当前工作侧重于实验验证，且均经过一定的简化，未来可改变游戏设计，改为对称对抗环境，从两人零和博弈开始逐步深入探讨强化学习与博弈论的结合，验证并感受主流智能体博弈方法。

（2）**更丰富的策略多样性机制：**当前游戏博弈性不高，考虑到时间因素，游戏偏简单，未来可以引入远程攻击和翻滚等角色技能，以及长线运营、协作任务等更复杂的游戏元素。

（3）**扩展到更大规模场景：**当前实验限于 4 人混战，可进行扩展，支持 n vs n vs $\dots$ n 的可配置阵容的对抗机制，方便进行更多的实验和测试。

结论

本文以 2D 多智能体竞技场博弈对抗为研究对象，完成了从环境构建到强化学习工程方法探索到博弈训练方法对比的完整研究流程。在环境层面，基于 Godot 引擎构建了《Tiny Swords》2D 多智能体竞技场博弈环境，设计了 174 维分段式观测空间和多层奖励塑形系统，并通过系统的消融实验确定了最优配置方案。在算法层面，采用 PPO 实现了单智能体训练，并引入了课程学习等技巧提高智能体表现，采用 IPPO 算法实现了多智能体同步训练。在博弈层面，提出了迭代自博弈训练框架，在不引入集中式 Critic 的前提下驱动多智能体策略持续对抗进化，实验观察到了测试对比了 IPPO、简化联盟训练和虚拟博弈三种博弈训练方法。

本课题的贡献在于提供了一套从环境构建到博弈训练的完整实验方案，对比了针对本课题特定环境的多智能体情况下主流博弈训练方法的差异性，为多智能体博弈对抗的进一步研究提供了实践基础和参考经验。

参考文献

- [1] SILVER D, HUANG A, MADDISON C J, et al. Mastering the game of go with deep neural networks and tree search[J/OL]. Nature, 2016, 529(7587): 484-489. <https://doi.org/10.1038/nature16961>.
- [2] VINYALS O, BABUSCHKIN I, CZARNECKI W M, et al. Grandmaster level in starcraft ii using multi-agent reinforcement learning[J/OL]. Nature, 2019, 575(7782): 350-354. <https://doi.org/10.1038/s41586-019-1724-z>.
- [3] OPENAI, :, BERNER C, et al. Dota 2 with large scale deep reinforcement learning [A/OL]. 2019. arXiv: 1912.06680. <https://arxiv.org/abs/1912.06680>.
- [4] CAMPBELL M, HOANE A, HSIUNG HSU F. Deep blue[J/OL]. Artificial Intelligence, 2002, 134(1): 57-83. <https://www.sciencedirect.com/science/article/pii/S0004370201001291>. DOI: [https://doi.org/10.1016/S0004-3702\(01\)00129-1](https://doi.org/10.1016/S0004-3702(01)00129-1).
- [5] HERNANDEZ-LEAL P, KARTAL B, TAYLOR M E. A survey and critique of multi-agent deep reinforcement learning[J/OL]. Autonomous Agents and Multi-Agent Systems, 2019, 33(6): 750-797. <https://doi.org/10.1007/s10458-019-09421-1>.
- [6] RASHID T, SAMVELYAN M, DE WITT C S, et al. QMIX: Monotonic value function factorisation for deep multi-agent reinforcement learning[C]//Proceedings of the 35th International Conference on Machine Learning (ICML): Vol. 80. PMLR, 2018: 4295-4304.
- [7] YU C, VELU A, VINITSKY E, et al. The surprising effectiveness of ppo in cooperative, multi-agent games[A/OL]. 2022. arXiv: 2103.01955. <https://arxiv.org/abs/2103.01955>.
- [8] DE WITT C S, GUPTA T, MAKOVICHUK D, et al. Is independent learning all you need in the starcraft multi-agent challenge?[A/OL]. 2020. arXiv: 2011.09533. <https://arxiv.org/abs/2011.09533>.
- [9] WATKINS C J C H, DAYAN P. Q-learning[J/OL]. Machine Learning, 1992, 8(3): 279-292. DOI: 10.1007/BF00992698.
- [10] CYBENKO G. Approximation by superpositions of a sigmoidal function[J/OL]. Mathematics of Control, Signals and Systems, 1989, 2(4): 303-314. DOI: 10.1007/BF02551274.
- [11] HORNIK K, STINCHCOMBE M, WHITE H. Multilayer feedforward networks are universal approximators[J/OL]. Neural Networks, 1989, 2(5): 359-366. DOI: 10.1016/0893-6080(89)90020-8.
- [12] MNIH V, KAVUKCUOGLU K, SILVER D, et al. Human-level control through deep reinforcement learning[J/OL]. Nature, 2015, 518(7540): 529-533. DOI: 10.1038/nature14236.
- [13] SCHULMAN J, WOLSKI F, DHARIWAL P, et al. Proximal policy optimization algorithms[A]. 2017.

- [14] SCHULMAN J, MORITZ P, LEVINE S, et al. High-dimensional continuous control using generalized advantage estimation[C]//International Conference on Learning Representations (ICLR). 2016.
- [15] 司鹏搏, 吴兵, 杨睿哲, 等. 基于多智能体深度强化学习的无人机路径规划[J/OL]. 北京工业大学学报, 2023, 49(4): 449-458. <https://journal.bjut.edu.cn/bjgydxxb/cn/article/doi/10.11936/bjutxb2022080007>.
- [16] 姚悦, 吉明佳, 杨霄. 基于改进PPO算法的自动驾驶技术研究[J]. 电子测量技术, 2023, 46(8): 162-168.
- [17] 万宇航, 朱子璐, 钟春富, 等. 基于改进PPO算法的机械臂动态路径规划[J]. 系统仿真学报, 2025, 37(6): 1462-1473.
- [18] GU Z, GAO L, MA H, et al. Safe-state enhancement method for autonomous driving via direct hierarchical reinforcement learning[J/OL]. IEEE Transactions on Intelligent Transportation Systems, 2023, 24(9): 9966-9983. DOI: 10.1109/TITS.2023.3271642.
- [19] BENGIO Y, LOURADOUR J, COLLOBERT R, et al. Curriculum learning[J]. ACM, 2009.
- [20] TSENG W C, WANG T H, YEN-CHEN L, et al. Offline multi-agent reinforcement learning with knowledge distillation[J/OL]. Advances in Neural Information Processing Systems 35, 2022. <https://api.semanticscholar.org/CorpusID:254128961>.
- [21] BELLEMARE M G, NADDAF Y, VENESS J, et al. The arcade learning environment: An evaluation platform for general agents[J]. Journal of Artificial Intelligence Research, 2013, 47: 253-279.
- [22] BROCKMAN G, CHEUNG V, PETTERSSON L, et al. Openai gym[A]. 2016. arXiv:1606.01540.
- [23] SAMVELYAN M, RASHID T, DE WITT C S, et al. The starcraft multi-agent challenge [A/OL]. 2019. arXiv: 1902.04043. <https://arxiv.org/abs/1902.04043>.
- [24] MORDATCH I, ABBEEL P. Emergence of grounded compositional language in multi-agent populations[A/OL]. 2018. arXiv: 1703.04908. <https://arxiv.org/abs/1703.04908>.
- [25] JULIANI A, BERGES V P, TENG E, et al. Unity: A general platform for intelligent agents[A/OL]. 2020. arXiv: 1809.02627. <https://arxiv.org/abs/1809.02627>.
- [26] BEECHING E, DIBANGOYE J, SIMONIN O, et al. Godot reinforcement learning agents[A]. 2021.
- [27] BROWN G W. Iterative solution of games by fictitious play[M]//KOOPMANS T C. Activity Analysis of Production and Allocation. New York: John Wiley & Sons, 1951: 374-376.
- [28] HEINRICH J, SILVER D. Deep reinforcement learning from self-play in imperfect-information games[A/OL]. 2016. arXiv: 1603.01121. <https://arxiv.org/abs/1603.01121>.

- [29] LANCTOT M, ZAMBALDI V, GRUSLYS A, et al. A unified game-theoretic approach to multiagent reinforcement learning[A/OL]. 2017. arXiv: 1711.00832. <https://arxiv.org/abs/1711.00832>.
- [30] DEVIDZE R. Reward design for reinforcement learning agents[A/OL]. 2025. arXiv: 2503.21949. <https://arxiv.org/abs/2503.21949>.
- [31] NG A, HARADA D, RUSSELL S J. Policy invariance under reward transformations: Theory and application to reward shaping[C/OL]//International Conference on Machine Learning. 1999. <https://api.semanticscholar.org/CorpusID:5730166>.
- [32] DEVLIN S, KUDENKO D. Dynamic potential-based reward shaping[C]//Proceedings of the 11th International Conference on Autonomous Agents and Multiagent Systems (AAMAS). 2012: 433-440.
- [33] DEVLIN S, KUDENKO D. Theoretical considerations of potential-based reward shaping for multi-agent systems[C]//Proceedings of the 10th International Conference on Autonomous Agents and Multiagent Systems (AAMAS). 2011: 225-232.
- [34] NG A Y, RUSSELL S. Algorithms for inverse reinforcement learning[C]//Proceedings of the 17th International Conference on Machine Learning (ICML). 2000: 663-670.
- [35] ZIEBART B D, MAAS A, BAGNELL J A, et al. Maximum entropy inverse reinforcement learning[C]//Proceedings of the 23rd AAAI Conference on Artificial Intelligence. 2008: 1433-1438.
- [36] WULFMEIER M, ONDRUSKA P, POSNER I. Maximum entropy deep inverse reinforcement learning[A]. 2015. arXiv: 1507.04888.
- [37] SELF R, ABUDIA M, MAHMUD S M N, et al. Model-based inverse reinforcement learning for deterministic systems[J]. Automatica, 2022, 140: 110242.
- [38] HADFIELD-MENELL D, MILLI S, ABBEEL P, et al. Inverse reward design[A/OL]. 2020. arXiv: 1711.02827. <https://arxiv.org/abs/1711.02827>.
- [39] PATHAK D, AGRAWAL P, EFROS A A, et al. Curiosity-driven exploration by self-supervised prediction[A/OL]. 2017. arXiv: 1705.05363. <https://arxiv.org/abs/1705.05363>.
- [40] BURDA Y, EDWARDS H, STORKEY A, et al. Exploration by random network distillation[A/OL]. 2018. arXiv: 1810.12894. <https://arxiv.org/abs/1810.12894>.
- [41] DEVIDZE R, KAMALARUBAN P, SINGLA A. Exploration-guided reward shaping for reinforcement learning under sparse rewards[C]//Advances in Neural Information Processing Systems (NeurIPS): Vol. 35. 2022: 5829-5842.
- [42] XIE T, ZHAO S, WU C H, et al. Text2reward: Reward shaping with language models for reinforcement learning[A/OL]. 2024. arXiv: 2309.11489. <https://arxiv.org/abs/2309.11489>.
- [43] 王琦, 杨毅远, 江季. EASYRL: 强化学习教程[M]. 人民邮电出版社, 2022: 1-2.

- [44] SUTTON R S, MCALLESTER D, SINGH S, et al. Policy gradient methods for reinforcement learning with function approximation[C]//Advances in Neural Information Processing Systems (NeurIPS): Vol. 12. MIT Press, 1999: 1057-1063.
- [45] SCHULMAN J, LEVINE S, ABBEEL P, et al. Trust region policy optimization[C]//Proceedings of the 32nd International Conference on Machine Learning (ICML): Vol. 37. PMLR, 2015: 1889-1897.
- [46] GAO Y, TONI F. Potential based reward shaping for hierarchical reinforcement learning [C]//IJCAI'15: Proceedings of the 24th International Conference on Artificial Intelligence. Buenos Aires, Argentina: AAAI Press, 2015: 3504–3510.
- [47] SMIRNOV I, GU S. Rlbenchnet: The right network for the right reinforcement learning task[A/OL]. 2025. arXiv: 2505.15040. <https://arxiv.org/abs/2505.15040>.
- [48] DENG X, LI N, MGUNI D, et al. On the complexity of computing markov perfect equilibrium in general-sum stochastic games[J/OL]. National Science Review, 2022, 10 (1). <http://dx.doi.org/10.1093/nsr/nwac256>.
- [49] DASKALAKIS C, FOSTER D J, GOLOWICH N. Independent policy gradient methods for competitive reinforcement learning[A/OL]. 2021. arXiv: 2101.04233. <https://arxiv.org/abs/2101.04233>.

第A章 附录A 实验补充数据

表 A-1: 有效位掩码对比：每回合总得分均值，5次独立运行均值，100 万步

方案	智能体0	智能体1	智能体2	智能体3	总平均
无有效掩码	75.2	72.5	65.9	41.0	254.7
有有效掩码	77.6	73.3	68.2	60.8	279.8

注：无有效掩码方案将缺失槽位填0，有有效掩码方案为每个槽位增加valid标志位。有效掩码总得分提高 9.9%，其中智能体3改善最显著（48.3%），智能体2次之（3.5%）。

表 A-2: 最近球与所有球势能方案每回合球收集奖励均值对比，10 次独立运行均值，100 万步

方案	智能体0	智能体1	智能体2	智能体3	总平均
仅最近球	50.1	43.6	54.3	44.1	192.1
所有球	31.2	26.1	28.7	28.8	114.8

注：最近球方案仅对视野内距离最近的奖励球计算势能，所有球方案对视野内全部奖励球叠加势能。最近球方案总平均比所有球方案高出约 67%，且各智能体均显著优于所有球方案。两种方案均不改变最优策略，但多球势能叠加在球间中点区域形成势能抵消，减弱引导密度。

表 A-3: 不同奖励塑形方案每回合球收集奖励均值对比，v3，含所有方案，5次独立运行均值，100 万步

方案	智能体0	智能体1	智能体2	智能体3	总平均
线性势函数	100.5	89.0	91.4	98.4	379.4
指数势函数	81.2	76.0	77.8	75.0	310.1
反比例势函数	100.6	93.4	91.7	94.6	380.2
距离奖励	100.7	97.3	94.0	98.1	390.1
稀疏	56.6	55.2	61.2	61.1	234.0

注：五种方案在相同单智能体环境下训练 100 万步，5 次独立运行取均值。线性函数为 $\Phi(s) = -\frac{R_{\text{ball}}}{r_{\text{vision}}}d + R_{\text{ball}}$ ，指数函数为 $\Phi(s) = R_{\text{ball}}e^{-d/r_{\text{vision}}}$ ，反比例函数为 $\Phi(s) = R_{\text{ball}}\frac{r_{\text{vision}}}{d+r_{\text{vision}}}$ ，距离奖励为 $\Delta R = d_t - d_{t-1}$ （靠近加分、远离减分），稀疏方案仅通过捡球得分学习，不使用任何奖励塑形。距离奖励方案得分最高，不过线性势函数与反比例势函数表现均与之接近，且势函数满足 PBRs 最优策略不变性，线性势函数被选为最终方案。

表 A-4: 撞墙策略综合对比表

(a) 稀疏惩罚力度对比

惩罚值	智能体0	智能体1	智能体2	智能体3	撞墙次数
0.05	73.9	65.3	66.5	49.1	161.5
0.5	58.8	60.8	52.3	37.6	114.8
5.0	20.8	22.6	22.3	-21.8	16.7

注：惩罚力度 0.5 在吃球得分与撞墙抑制之间取得最佳平衡（撞墙减少 29%，得分基本持平）。5.0 惩罚显著减少撞墙但导致过度保守。

(b) 障碍物塑形方案对比

方案	智能体0	智能体1	智能体2	智能体3	撞墙次数
稀疏惩罚 (0.5)	58.8	60.8	52.3	37.6	114.8
线性势函数	59.9	54.7	51.0	37.8	52.6
反比例势函数	60.8	55.3	53.0	30.1	54.7
距离惩罚	57.0	47.6	49.0	31.6	44.7

注：稀疏惩罚采用 0.5 碰撞惩罚无奖励塑形。线性势函数 $\Phi(s) = \frac{R_{\text{wall}}}{r_{\text{vision}}} d_{\text{min}} - R_{\text{wall}}$ ，反比例势函数 $\Phi(s) = -\frac{1}{d_{\text{min}} + \epsilon}$ ，距离惩罚每步给予 $-\frac{1}{d_{\text{min}} + \epsilon}$ 。距离惩罚撞墙最少但得分也最低；线性势函数综合表现最佳且满足PBRs最优策略不变性。

表 A-5: 吃球任务（S1）网络架构对比（单智能体，5次独立运行均值，100 万步）

网络架构	智能体0	智能体1	智能体2	智能体3	总得分	吃球得分
MLP	213.6	218.0	202.0	228.7	862.4	852.1
Segmented MLP	251.8	222.1	233.5	233.3	940.6	934.8
GRU-MLP（隐藏层96）	243.5	212.0	196.5	242.4	894.4	884.0
GRU-MLP（隐藏层128）	238.1	207.3	222.3	237.2	905.0	898.2

注：四种网络架构在 S1 吃球课程环境中各训练 50 万步。吃球任务相对简单，各架构得分接近，未拉开显著差距；Segmented MLP 得分最高（940.6），GRU-MLP 隐藏层 128 维度次之（905.0）。

表 A-6: Sarl 战斗任务网络架构对比（单智能体，5 次独立运行均值）

网络架构	总得分	攻击次数	造成伤害	承受伤害	击杀数	死亡数	撞墙次数
MLP	775.2	69.5	77.6	26.7	38.4	2.2	3.4
Segmented MLP	714.6	64.1	73.0	26.2	36.1	2.2	7.7
GRU-MLP（隐藏层96）	582.9	64.1	61.3	22.7	30.3	1.8	23.6
GRU-MLP（隐藏层128）	578.5	79.0	60.6	22.7	30.0	1.8	15.0

表 A-7: 课程学习阶段配置表

阶段	环境名称	新增要素	训练步数	累计步数	输出模型
S1	s1-no-wall-for-ball	仅吃球	50万	50万	mlp_s1
S2	s2-wall-and-ball	障碍物	100万	150万	mlp_s2
S3	s3-enemy-and-ball	敌人，智能体在中心区域内出生	200万	350万	mlp_s3
S4	s4-enemy-and-wall-and-ball	障碍物	100万	450万	mlp_s4
Main	godot-game	更改出生点	50万	500万	ppo_a

注：每个阶段 4 个智能体（agent_0~agent_3）独立训练，使用各 build 目录和端口偏移，确保完全并行。每阶段加载上一阶段模型，采用 MLP 网络 + Optuna 搜索后的超参数。累计训练步数 500 万步。

致 谢

致谢部分感谢在毕业设计过程中给予帮助的老师、同学和家人。

首先，我要衷心感谢我的指导教师刘春阳老师在选题、研究和论文撰写过程中给予的悉心指导。

.....